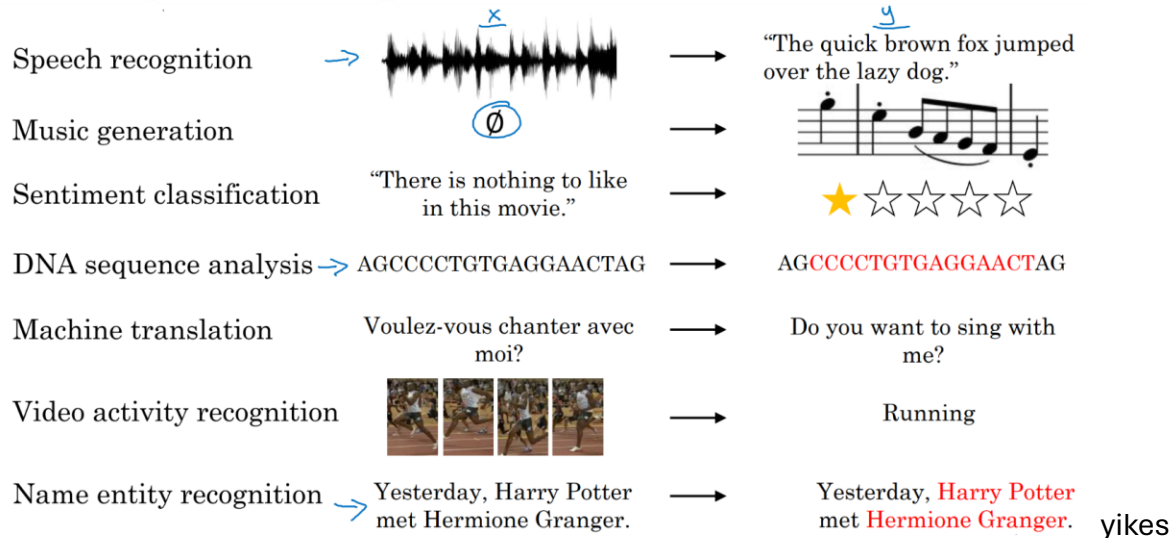


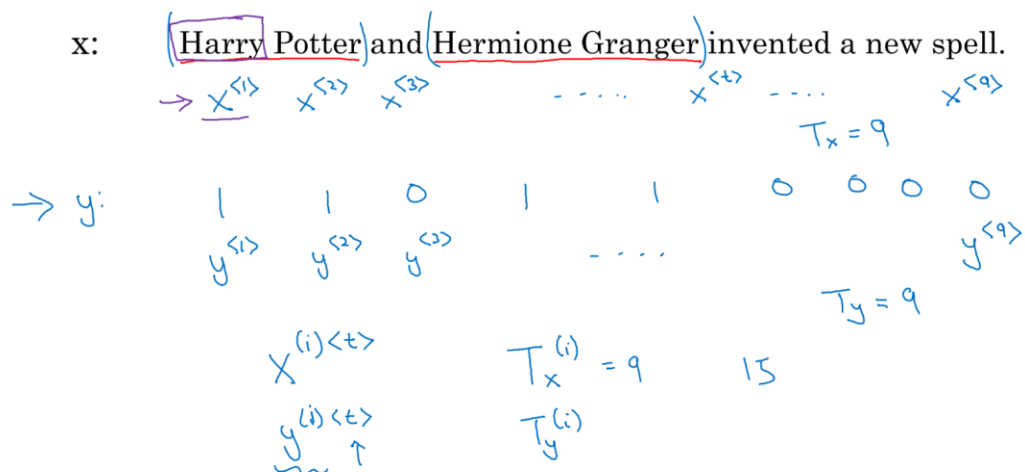
Why Sequence Models?

- Examples of sequence data
 - Sometimes both x and y are sequences, sometimes only one or the other

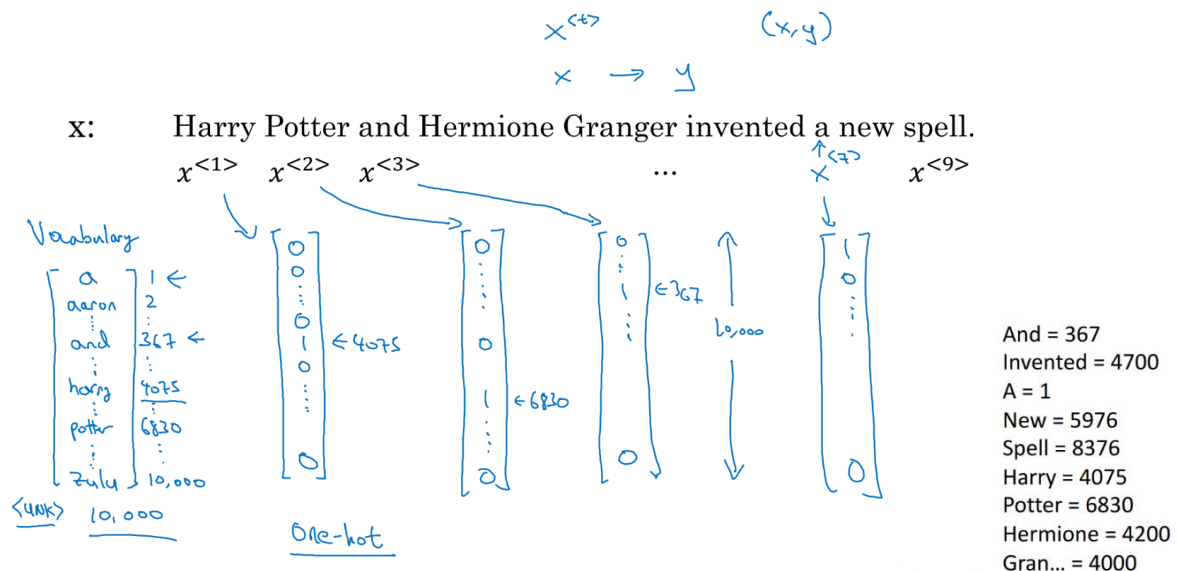


Notation

- Example (yikes outdated reference)
 - Named-entity recognition: input a sentence and find names (people, country)
 - More sophisticated representations tell you where the start and end are
 - Here, nine sets of features $x^{<i>}$ to represent the nine words, nine output values $y^{<i>}$ = 0 or 1, T_x = length of input sequence, T_y = length of output sequence)
 - $T_x^{(i)}$ is the input sequence length for training example i
 - $y^{(i)<t>}$ is the t^{th} element in the output sequence of i^{th} training example

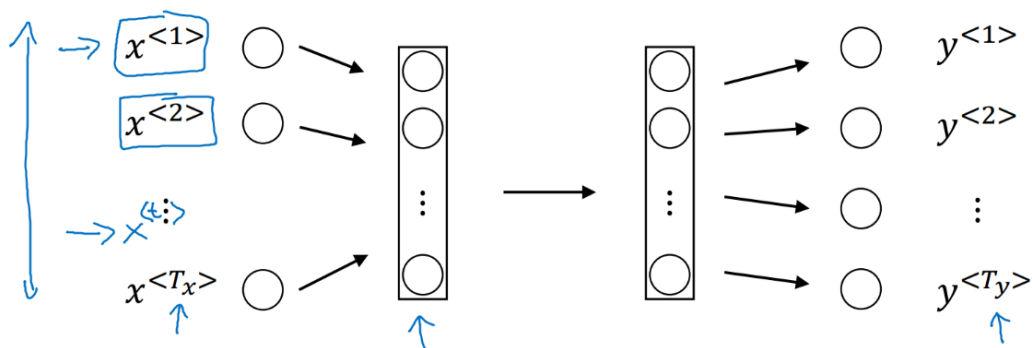


- Representing words
 - Vocabulary (dictionary) catalog of words available
 - Use one-hot representations for each of these words
 - Words not in dictionary: create a token called Unknown Word <UNK>



Recurrent Neural Network Model

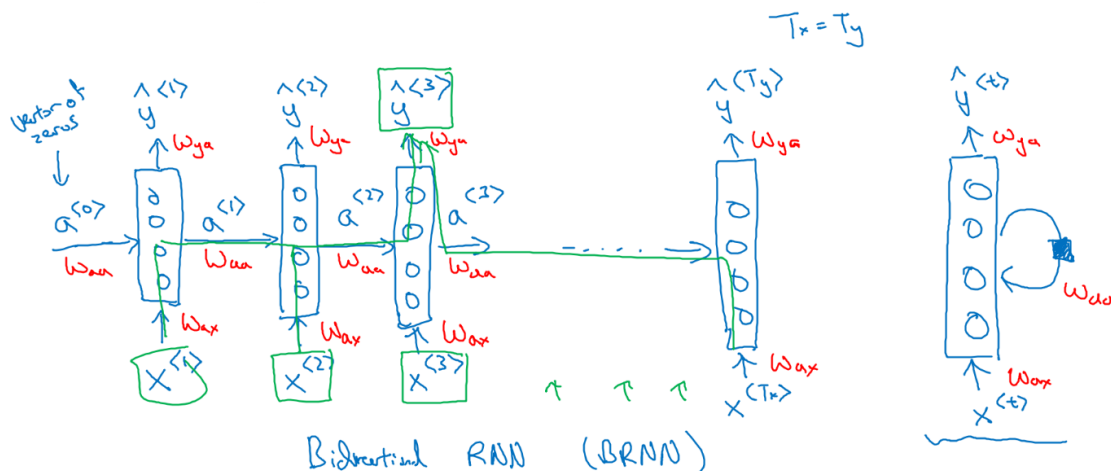
- Learn mapping from x to y (sequence data/output)
- Why not a standard network?



- Problems:
 - Inputs, outputs can be different lengths in different examples
 - Doesn't share features learned across different positions of text (we want things learned in one part to generalize quickly to other parts)

- Recurrent Neural Networks

- Feed 1st word into NN layer and predict output (name or not). When reading second word, instead of just predicting $y^{<2>}$ using only $x^{<2>}$, it also gets the activation value at time step 1
- Some made up initialize activations, usually a vector of 0s or sometimes random
- Parameters governing connection from $x^{<1>}$ to the hidden layer will be some set of parameters W_{ax} used for every time step
- The activations (horizontal connections) will be governed by some set of parameters W_{aa} used for every time step
- W_{ya} governs the output predictions
- Weakness: only uses information that is earlier in the sequenced to make a prediction. There is a lot of information to be gained afterwards (bidirectional recurrent neural networks address this)

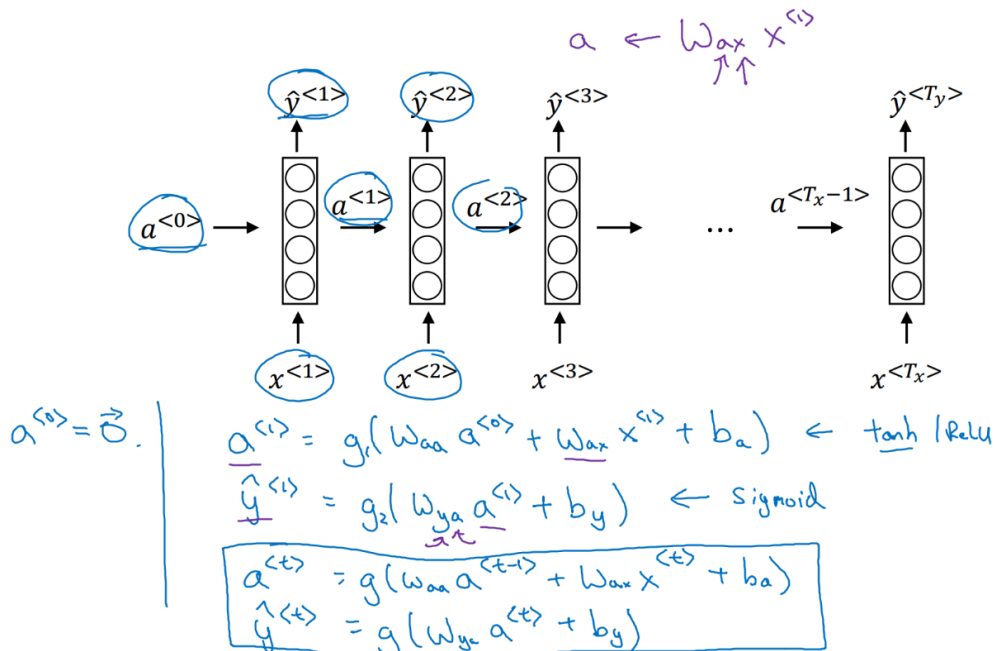


He said, "Teddy" Roosevelt was a great President."

He said, "Teddy" bears are on sale!"

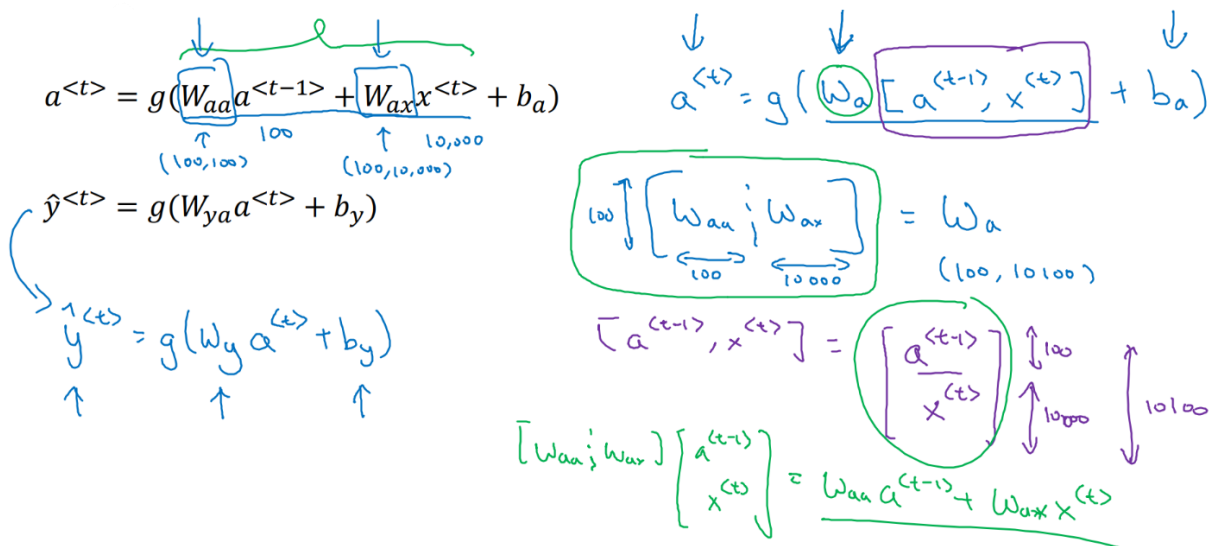
- Forward propagation

- Initialize $a^{<0>}$ as a vector of 0s
- $a^{<t>} = g_1(W_{aa} a^{<t-1>} + W_{ax} x^{<t>} + b_a)$ [tanh/ReLU]
- $\hat{y}^{<t>} = g_2(W_{ya} a^{<t>} + b_y)$ [sigmoid/softmax]



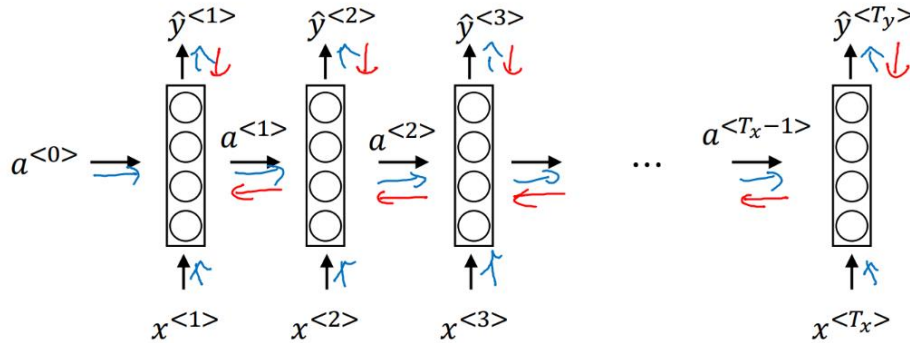
- Simplified RNN notation

- $a^{<t>} = g(W_{aa} a^{<t-1>} + W_{ax} x^{<t>} + b_a) = g(W_a [a^{<t-1>}, x^{<t>}] + b_a)$
 - Compress into one parameter matrix
 - $W_a = [W_{aa}; W_{ax}]$ $[a^{<t-1>}, x^{<t>}] = [a^{<t-1>}, x^{<t>}]^T$ $\hat{y}^{<t>} = g(W_y a^{<t>} + b_y)$

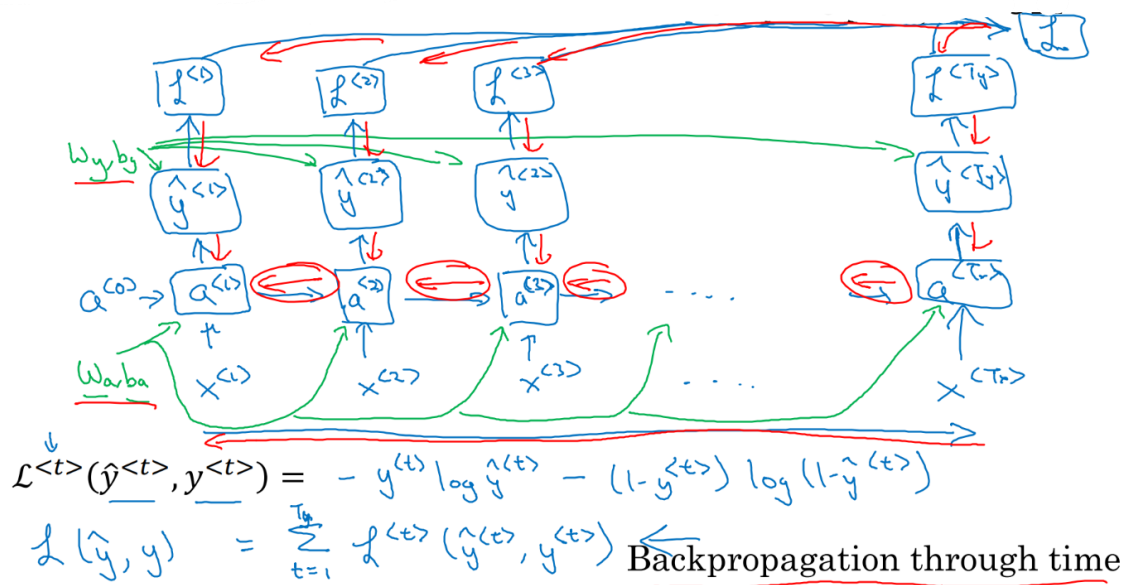


Backpropagation Through Time

- Most programming frameworks will automatically take care of backprop
- Forward propagation and backpropagation

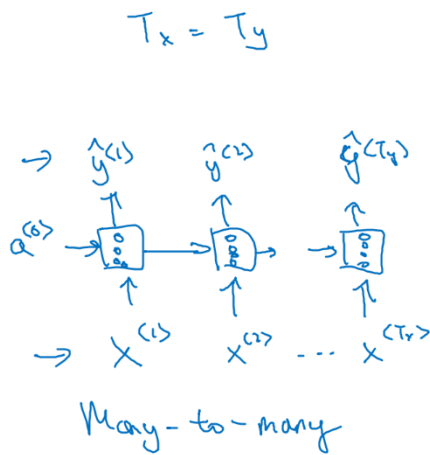
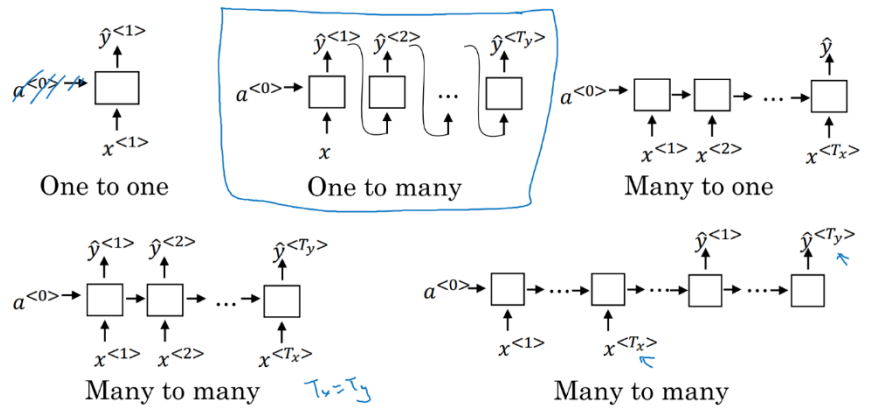


- Logistic (cross-entropy loss)
- Backpropagation right to left

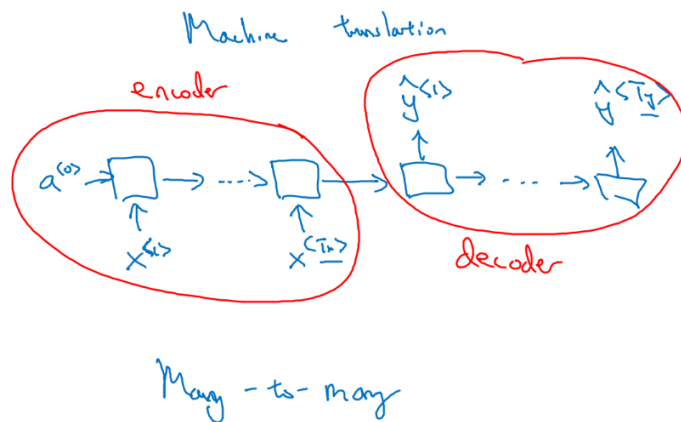
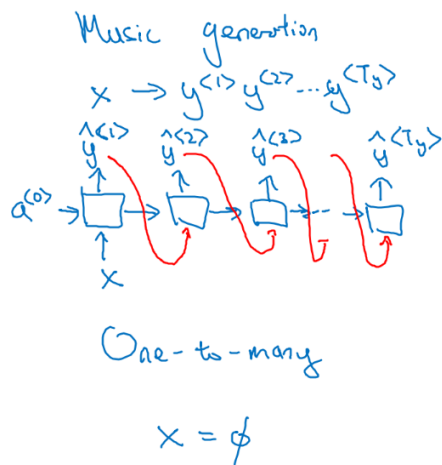
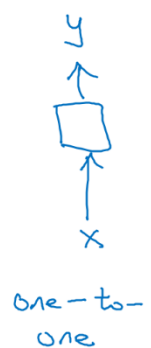
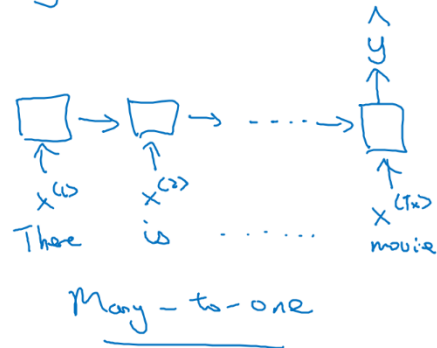


Different Types of RNNs

- One-to-one: standard generic NN
- One-to-many: music or sequence generation (given keyword e.g. genre)
- Many-to-one: sentiment classification (movie review text $\rightarrow$ like yes/no)
- Many-to-many: name-entity recognition, language translation (languages sometimes have a different number of words for the same sentence)



Sentiment classification
 $x = \text{text}$
 $y = 0/1 \quad 1 \dots 5$



Language Model and Sequence Generation

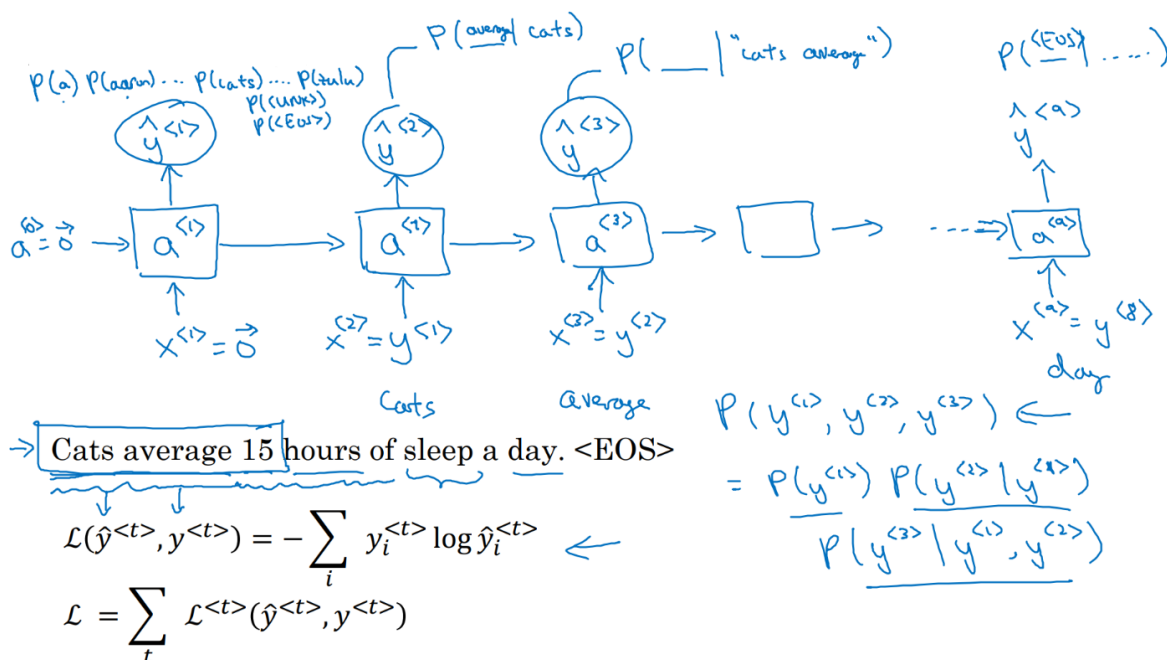
- What is language modeling?
- Homophones example:
 - Language model estimated the probability of sequence of words

$$P(\text{The apple and pair salad}) = 3.2 \times 10^{-13}$$

$$P(\text{The apple and pear salad}) = 5.7 \times 10^{-10}$$

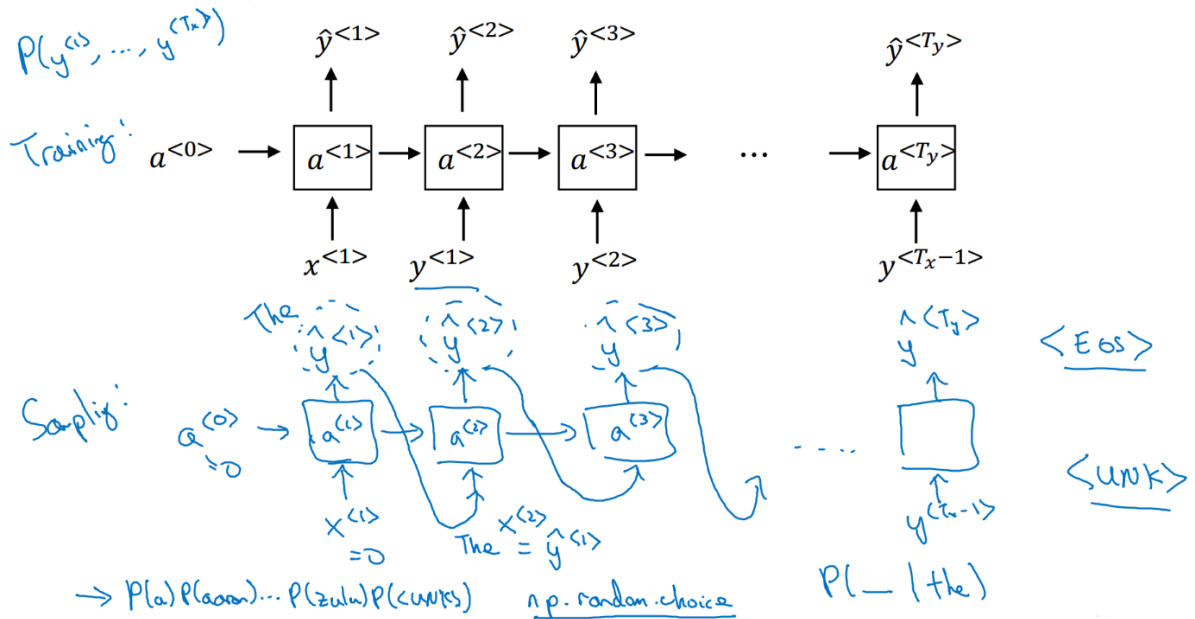
$$P(\text{Sentence}) = ? \quad P(y^{(1)}, y^{(2)}, \dots, y^{(T)})$$

- Language modelling with an RNN
 - Training set: large corpus of english text (tokenize)
 - Tokenize the sentence: form a vocabulary and map each word to one-hot vectors/indices of vocabulary. Can add a token for end of sentence <EOS> and punctuation
 - If there is a word not in your vocabulary, replace with unique token <UNK>
- RNN model
 - $a^{(1)}$ makes SoftMax prediction for 1st word
 - $a^{(2)}$ makes SoftMax prediction for 2nd word, and is given the first word
 - Softmax loss function
 - $P(\text{sentence}) = P(y^{(1)}) * P(y^{(2)} | y^{(1)}) * P(y^{(3)} | y^{(1)}, y^{(2)}) * \dots$



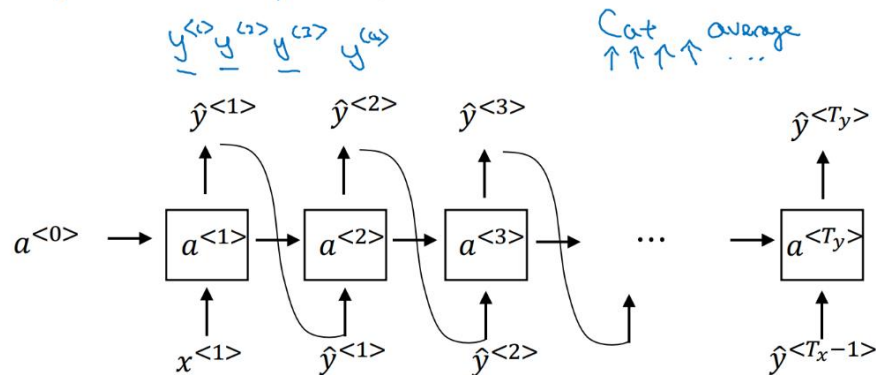
Sampling Novel Sequences

- Sampling a sequence from a trained RNN
 - Use `np.random.choice` to sample according to the distribution defined by the vector of probabilities for each word
 - Pass $\hat{y}^{<1>}$ as input to the next time step $\rightarrow \hat{y}^{<1>} = P(\text{__} | \hat{y}^{<1>})$
 - Keep sampling until you create an EOS token, or (if not in vocab) just decide to sample a certain number of words



- Character-level language model
 - Don't have to worry about unknown word tokens, but much longer sequences and much more computationally expensive to train

Vocabulary = [a, b, c, ..., z, _ , . , : , o , ... , q , A , ... , z]



- Sequence generation

News

President enrique peña nieto, announced
sench's sulk former coming football langston
paring.

"I was not at all surprised," said hich langston.

"Concussion epidemic", to be examined. ←

The gray football the told some and this has on
the uefa icon, should money as.

Shakespeare

The mortal moon hath her eclipse in love.

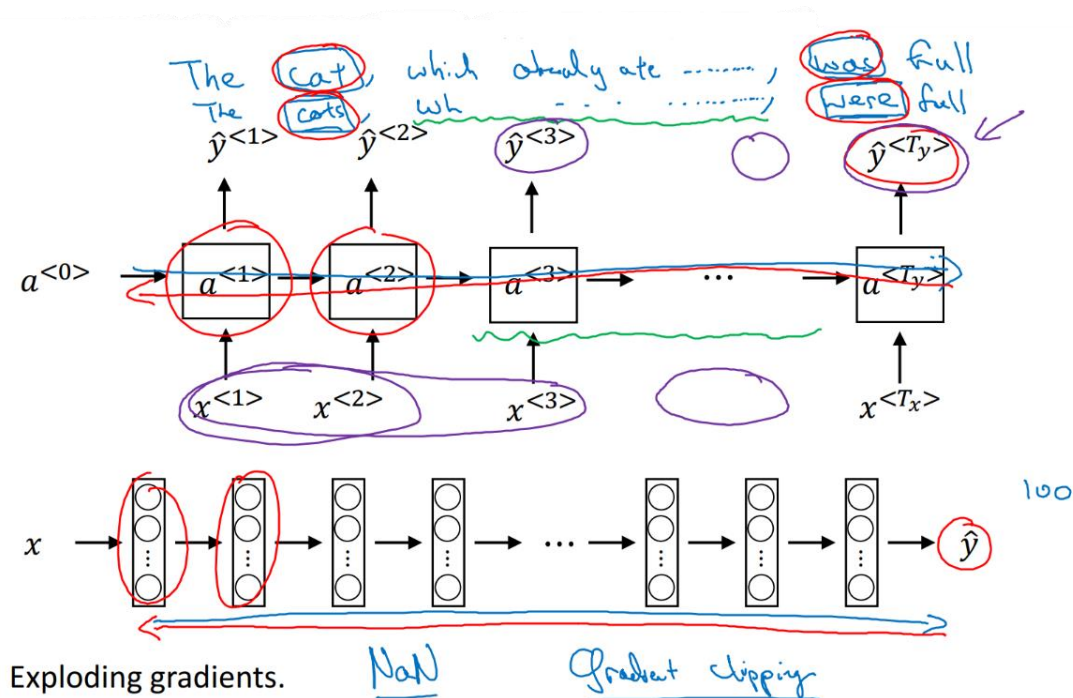
And subject of this thou art another this fold.

When besser be my love to me see sabl's.

For whose are ruse of mine eyes heaves.

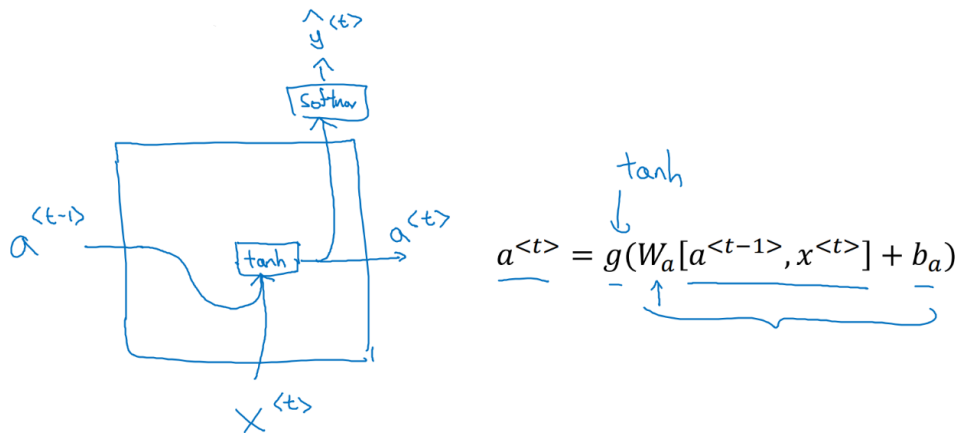
Vanishing Gradients with RNNs

- What if a word early in the sentence affects one much later? (e.g. split nouns and verbs)
- Basic RNN not very good at capturing long-term dependencies. Mainly local influences
- Very deep NN gradient from output has hard time propagating back to affect early layers
- E.g. memorize whether noun is singular or plural until the interrupting phrase is over
- Exploding gradients can also happen (easy to spot NaN overflow and apply gradient clipping). Vanishing is more common and much harder to solve



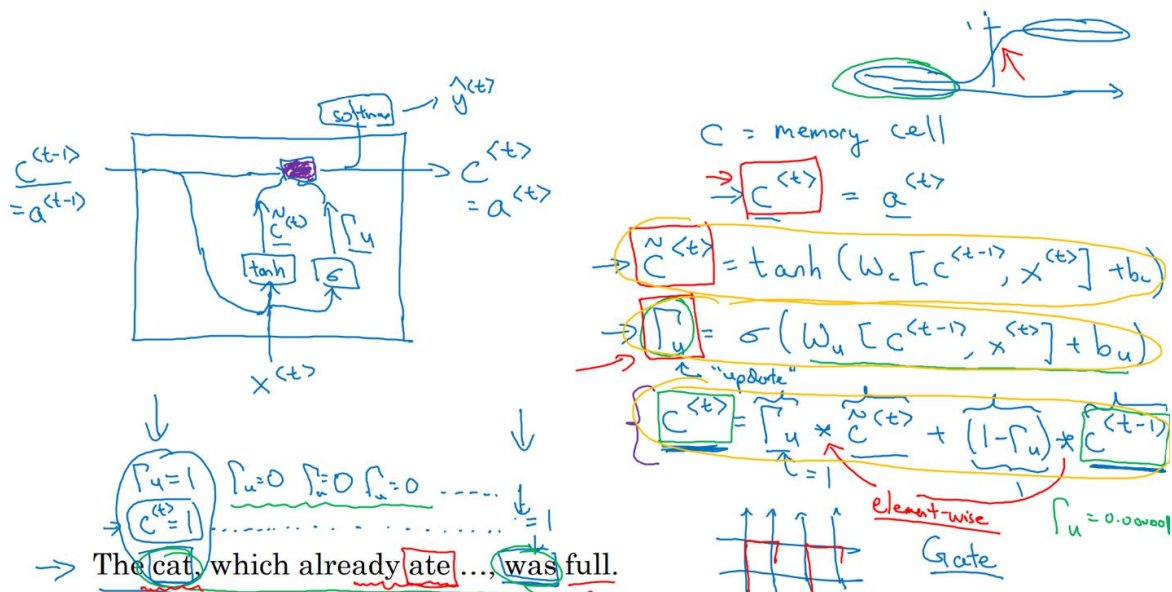
Gated Recurrent Unit (GRU)

- RNN unit



- GRU (simplified)

- c = memory cell
- For GRU, $c^{<t>} = a^{<t>}$ (will not be the case for LSTM)
- For each time step, we consider overwriting the value of $c^{<t>}$ with candidate $\tilde{c}^{<t>} = \tanh(W_c[c^{<t-1>}, x^{<t>}] + b_c)$
- Gate Γ_u (u = update) between 0 and 1, usually at 0 or 1 (sigmoid function) decides whether we update $c^{<t>}$. (0 means do not update, 1 means update)
- $\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$
- Memory cell formula: $c^{<t>} = \Gamma_u \tilde{c}^{<t>} + (1 - \Gamma_u) c^{<t-1>}$
 - Keeps $c^{<t-1>}$ when $\Gamma_u = 0$, updates to $\tilde{c}^{<t>}$ when $\Gamma_u = 1$



- Full GRU

- Another gate $\Gamma_r = \sigma W_r [c^{<t-1>}, x^{<t>}] + b_r$ (r = relevance)

$$\tilde{c}^{<t>} = \tanh(W_c [\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u [c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r [c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

LSTM (Long Short Term Memory) Unit

- GRU and LSTM

- Separate gates in c: $\Gamma_f = \sigma W_f [a^{<t-1>}, x^{<t>}] + b_f$ (f = forget)
- Output gates: $\Gamma_o = \sigma W_o [a^{<t-1>}, x^{<t>}] + b_o$
- Memory cell formula: $c^{<t>} = \Gamma_u * c^{<t>} + \Gamma_f c^{<t-1>}$
- Activation: $a^{<t>} = \Gamma_o \tanh c^{<t>}$

GRU

$$\tilde{c}^{<t>} = \tanh(W_c [\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u [c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r [c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c [a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u [a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f [a^{<t-1>}, x^{<t>}] + b_f)$$

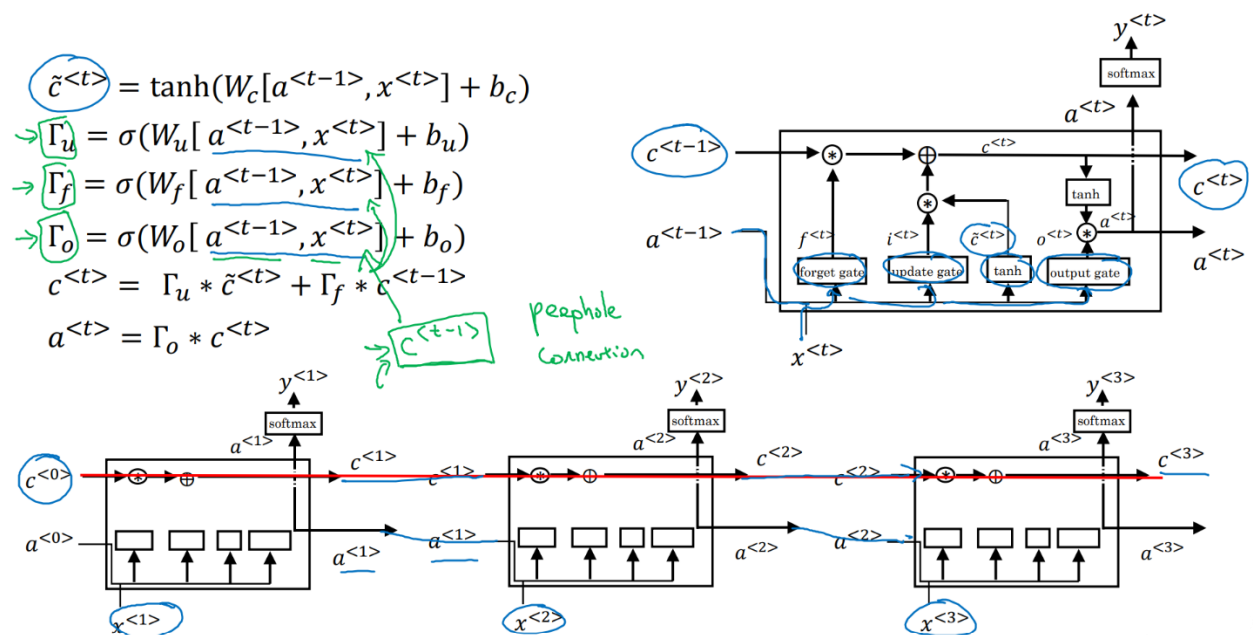
$$\Gamma_o = \sigma(W_o [a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

- LSTM in pictures

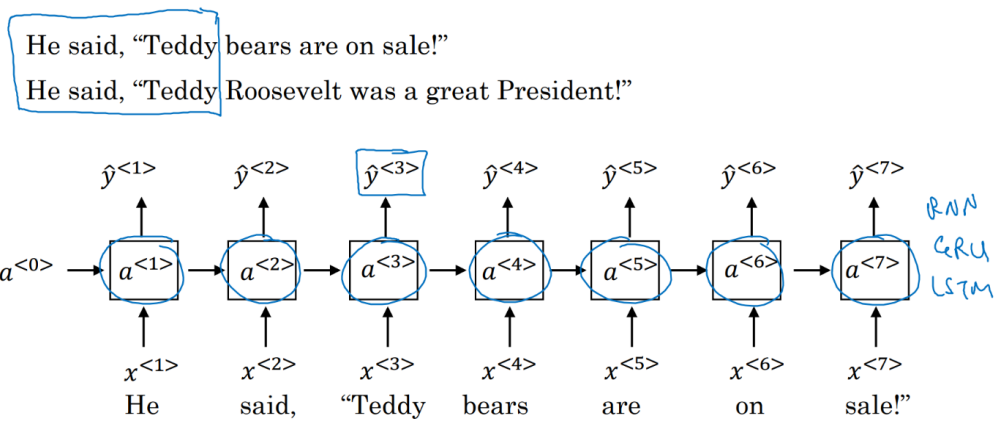
- Use $a^{<t-1>}$ and $x^{<t>}$ to compute all the gate values
- Go through tanh to compute $\tilde{c}^{<t>}$
- Hook them up in parallel. As long as you set the forget and update gates appropriately, it's relatively easy for the LSTM to pass some value $c^{<0>}$ all the way to the right
- Peephole connection: gate values also depend on $c^{<t-1>}$ (prev memory cell value)
- Note very element of $c^{<t-1>}$ can affect all elements of the gates (1st element of $c^{<t-1>}$ affects the 1st elements of the gates and so on)



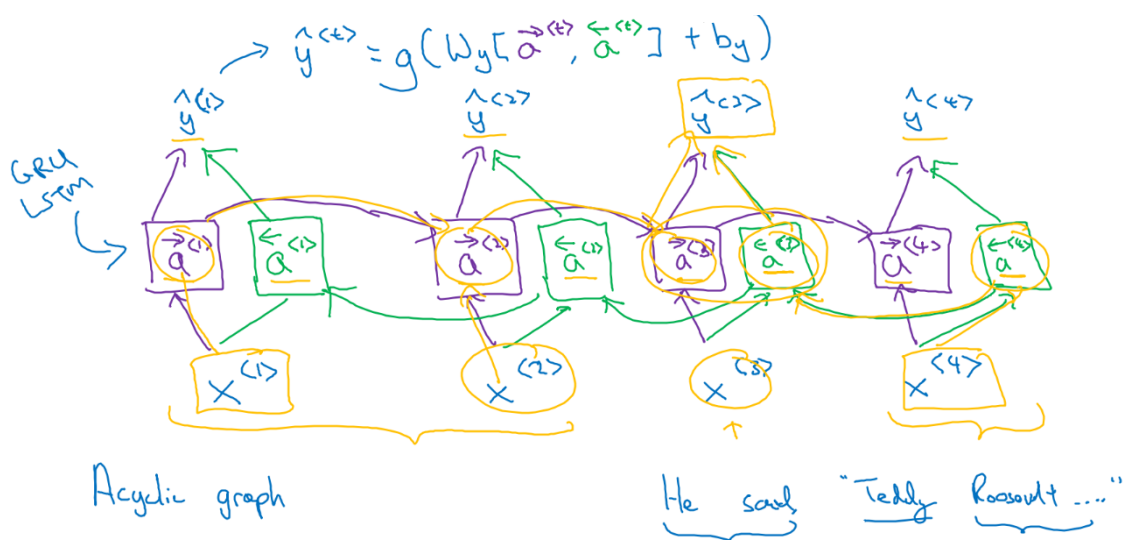
- LSTMs were made first, and GRUs were derived as a simplification of LSTMs
- LSTMs are more powerful and flexible

Bidirectional RNN

- Take information from both earlier and later in the sequence
- Getting information from the future
 - Can't just look at first part of sentence to tell if it's part of a person's name



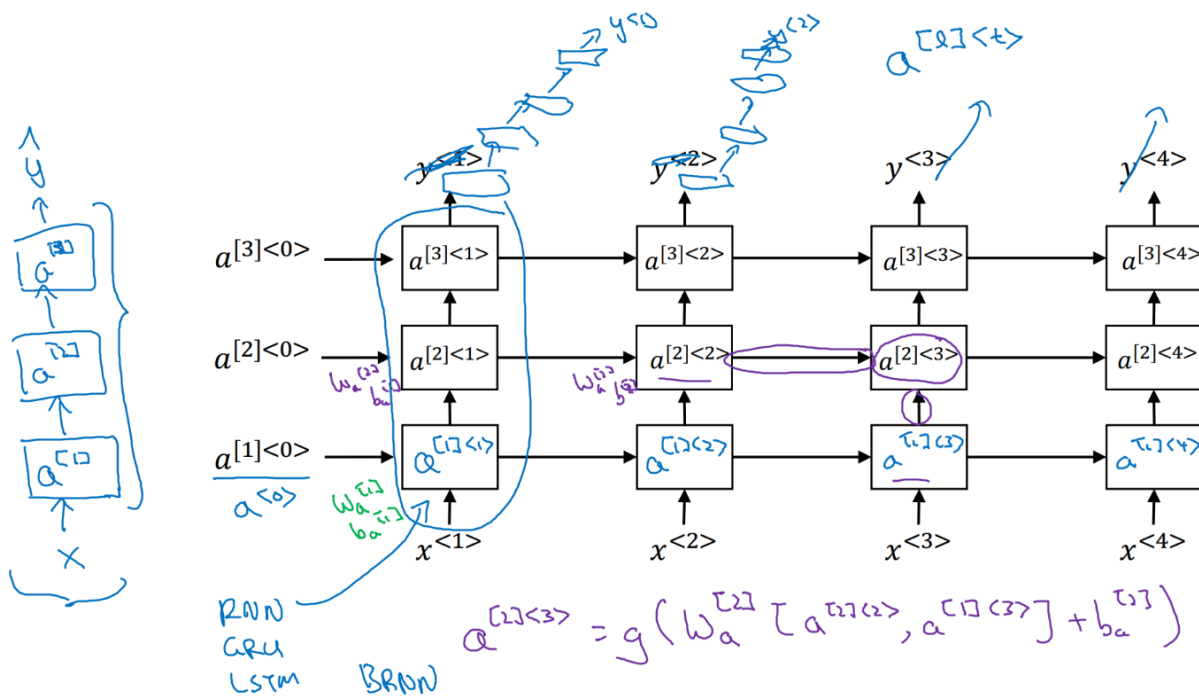
- Bidirectional RNN (BRNN)
 - Each hidden layer has a **forward hidden recurrent component** (taking the current x and previous activations to predict $\hat{y}$) and **backward recurrent hidden component** (taking activation of later layer and computing the previous layer's activation)
 - Acyclic graph
 - $\hat{y}^{<t>} = g(W_y[a^{<t>}_{\text{forward}}, a^{<t>}_{\text{backward}}] + b_y)$



- BRNN with LSTM is commonly used for natural language processing
- Disadvantage: need entire data seq before you can make predictions anywhere

Deep RNNs

- Stack multiple layers of RNNs together
- Unroll network on left in time
- $a^{[l]<t>}$ is the activation associated with layer l associated over time t
- Same parameters $W_a^{[l]}$ and $b_a^{[l]}$ are used for every computation at layer l
- 3 layers is already quite a lot for RNNs
- You may get rid of the outputs of each layer and replace with deep layers that are not connected horizontally
- The blocks don't have to be standard RNN. They can also be GRU block or LSTM blocks

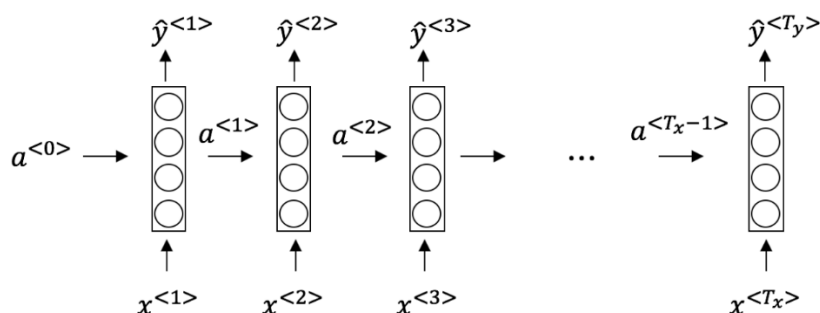


Suppose your training examples are sentences (sequences of words). Which of the following refers to the s^{th} word in the r^{th} training example?

$$x^{(s)<r>} \quad x^{<s>(r)} \quad x^{(r)<s>} \quad x^{<r>(s)}$$

We index into the r^{th} row first to get to the r^{th} training example (represented by parentheses), then the s^{th} column to get to the s^{th} word (represented by the brackets).

Consider this RNN:



This specific type of architecture is appropriate when:

$$T_x = T_y \quad T_x < T_y \quad T_x > T_y \quad T_x = 1 \quad \text{every input has an output}$$

Select the two tasks combination that could be addressed by a many-to-one RNN model architecture from the following:

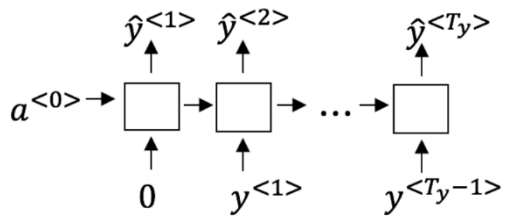
Task 1: Speech recognition. Task 2: Gender recognition from audio.

Task 1: Image classification. Task 2: Sentiment classification.

Task 1: Gender recognition from audio. Task 2: Image classification.

Task 1: Gender recognition from audio. Task 2: Movie review (positive/negative) classification.

Using this as the training model below, answer the following:



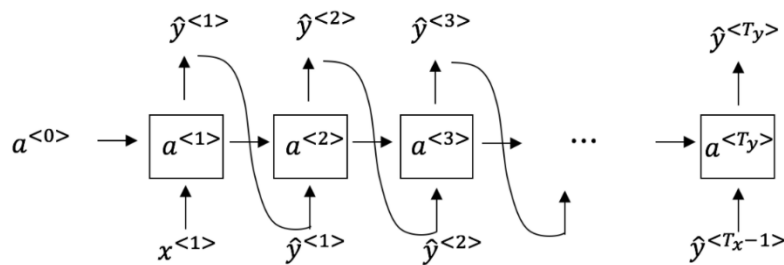
True/False: At the t^{th} time step the RNN is estimating $P(y^{<t>} \mid y^{<1>}, y^{<2>}, \dots, y^{<t-1>})$

False

True

In a training model we try to predict the next step based on knowledge of all prior steps

You have finished training a language model RNN and are using it to sample random sentences, as follows:



What are you doing at each time step t ?

(i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as $\hat{y}^{<t>}$. (ii) Then pass the ground-truth word from the training set to the next time-step.

(i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as $\hat{y}^{<t>}$. (ii) Then pass the ground-truth word from the training set to the next time-step.

(i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as $\hat{y}^{<t>}$. (ii) Then pass this selected word to the next time-step.

(i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as $\hat{y}^{<t>}$. (ii) Then pass this selected word to the next time-step.

Exploding gradients happen when large error gradients accumulate and result in very large updates to the NN model weights during training. These weights can become too large and cause an overflow, identified as NaN.

True/False: If you are training an RNN model, and find that your weights and activations are all taking on the value of NaN (“Not a Number”) then you have an exploding gradient problem.

False **True**

Exploding gradients happen when large error gradients accumulate and result in very large updates to the NN model weights during training. These weights can become too large and cause an overflow, identified as NaN.

Suppose you are training an LSTM. You have a 10000 word vocabulary and are using an LSTM with 100-dimensional activations $a^{<t>}$. What is the dimension of Γ_u at each time step?

1 **100** 300 10000

Γ_u is a vector of dimension equal o the number of hidden units in the LSTM

Sarah proposes to simplify the GRU by setting the update gate $\Gamma_u = 0$ (Always keep memory, never update). Ashley proposes to simplify the GRU by setting the relevance gate $\Gamma_r = 1$ (Always consider previous memory relevant). Which of these simplified models is more likely to work effectively without vanishing gradient problems when trained on very long input sequences?

GRU (For Reference)

$$\tilde{c}^{(t)} = \tanh (W_c[\Gamma_r * c^{(t-1)}, x^{(t)}] + b_c)$$

$$\Gamma_u = \sigma (W_u[c^{(t-1)}, x^{(t)}] + b_u) \qquad \Gamma_r = \sigma (W_r[c^{(t-1)}, x^{(t)}] + b_r)$$

$$c^{(t)} = \Gamma_u * \tilde{c}^{(t)} + (1 - \Gamma_u) * c^{(t-1)} \qquad a^{(t)} = c^{(t)}$$

Ashley's model (setting $\Gamma_r = 1$), because it retains the update gate Γ_u , which allows the network to control whether to keep or overwrite the memory cell.

Sarah's model (setting $\Gamma_u = 0$), because removing the update gate simplifies the architecture, making it easier for the relevance gate Γ_r to learn long-term dependencies.

Ashley's model (setting $\Gamma_r = 1$), because the relevance gate Γ_r is responsible for causing the vanishing gradient problem, so removing it solves the issue.

Sarah's model (setting $\Gamma_u = 0$), because the cell state will never change, ensuring gradients are perfectly preserved.

By retaining the update gate Γ_u , the model can choose when to update memory versus when to retain it. This allows the cell state $c^{(t)}$ to remain close to $c^{(t-1)}$ when needed, preserving long-term dependencies and gradients.

Here are the equations for the GRU and the LSTM:

GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

From these, we can see that the Update Gate and Forget Gate in the LSTM play a role similar to _____ and _____ in the GRU. What should go in the blanks?

Γ_u and $1 - \Gamma_u$ Γ_u and Γ_r $1 - \Gamma_u$ and Γ_u Γ_r and Γ_u

You have a pet dog whose mood is heavily dependent on the current and past few days' weather. You've collected data for the past 365 days on the weather, which you represent as a sequence as $x^{<1>}, \dots, x^{<365>}$. You've also collected data on your dog's mood, which you represent as $y^{<1>}, \dots, y^{<365>}$. You'd like to build a model to map from $x \rightarrow y$. Should you use a Unidirectional RNN or Bidirectional RNN for this problem?

Bidirectional RNN, because this allows the prediction of mood on day t to take into account more information.

Bidirectional RNN, because this allows backpropagation to compute more accurate gradients.

Unidirectional RNN, because the value of $y^{<t>}$ depends only on $x^{<1>}, \dots, x^{<t>}$, but not on $x^{<t+1>}, \dots, x^{<365>}$

Unidirectional RNN, because the value of $y^{<t>}$ depends only on $x^{<t>}$, and not other days' weather.

Word Representation

- O = one-hot
- Weakness: treats each word as a thing unto itself and doesn't allow an algorithm to easily generalize across words
- Inner product between any two different one-hot vectors is zero, and the Euclidian distance between any pair of these vectors is the same

$V = [a, aaron, \dots, zulu, <UNK>]$

$|V| = 10,000$

1-hot representation

Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$

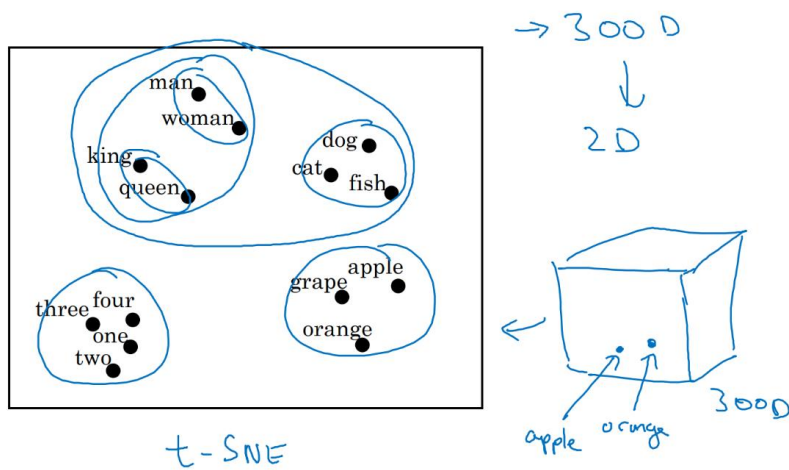
I want a glass of orange juice.
 I want a glass of apple juice.

- Featurized representation: **word embedding**
 - Create features and corresponding correlation metrics for each word
 - e = embedding for a word
 - Ex: orange and apple now have a similar feature the algorithm can use
 - The features won't always have factors with easy interpretation

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
size						
cost						
alike						
verb						

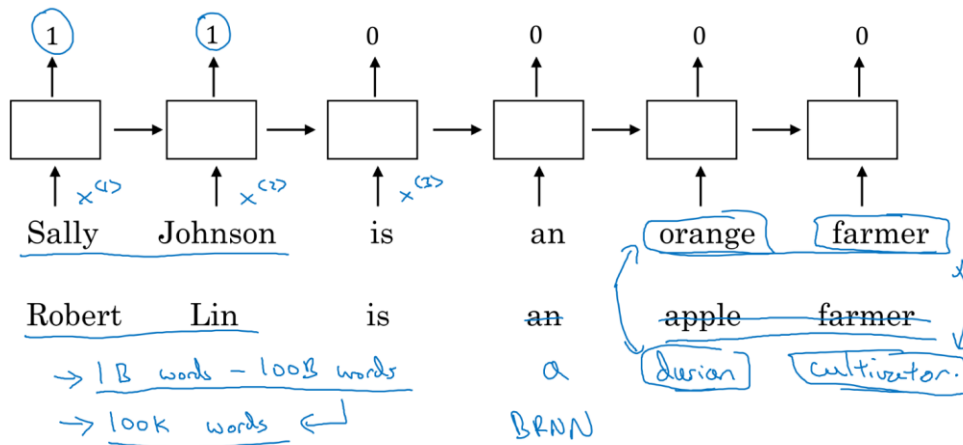
I want a glass of orange juice.
 I want a glass of apple juice.

- Visualizing word embeddings: embed in 2D space to visualize (t-SNE)



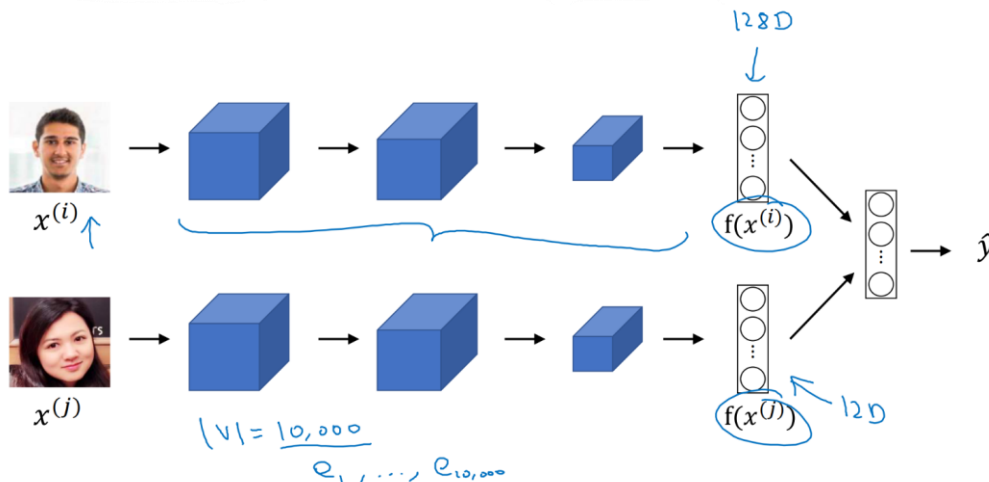
Using Word Embeddings

- Named entity recognition example
 - How to know if a name is a person or business name? Info from later in
 - Rare words not in training set? Algorithm can generalize based on what features the words have that the training set learned (durian is fruit, cultivator like farmer)
 - Transfer learning from huge amounts of unlabeled text online
 - Learn word embeddings and apply to named entity recognition task



- Transfer learning and word embeddings
 - 1. Learn word embeddings from large text corpus (1-100B words) (or download pre-trained embedding online)
 - 2. Transfer embedding to new task with smaller training set (say, 100k words)
 - 3. Optional: Continue to finetune the word embeddings with new data
 - Most useful when there was a lot of data for Task A, but little for Task B

- Relation to face encoding/embedding
 - Siamese network learns a 128-dimensional representation for different faces
 - Compare encodings to figure out if the pictures are of the same face
 - For face recognition, goal is to train a NN that can take as input any face picture and compute an encoding (even if the picture is new)
 - For learning word embeddings, we'll have a fixed vocabulary and learn a vector $e_1, \dots, e_n$ that learns a fixed encoding/embedding for each of the words in our vocabulary



Properties of Word Embedding

- Analogies
 - Given a relationship between two vectors (e.g. $e_{\text{man}} - e_{\text{woman}}$), what is the main difference in the pair? (E.g. gender)
 - Find another pair with the same relationship (e.g. $e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - e_{\text{queen}}$)

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97

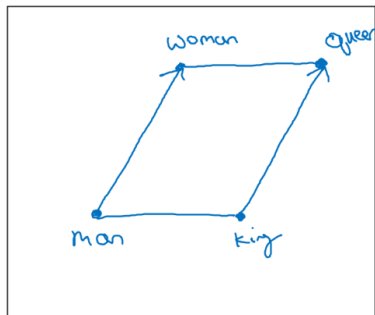
Handwritten notes illustrating word analogies:

$e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - e_{\text{queen}}$
 $e_{\text{man}} - e_{\text{woman}} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
 $e_{\text{king}} - e_{\text{queen}} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Man $\rightarrow$ Woman $\approx$ King $\rightarrow$? Queen
 $e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - e_{\text{queen}}$

- Analogies using word vectors

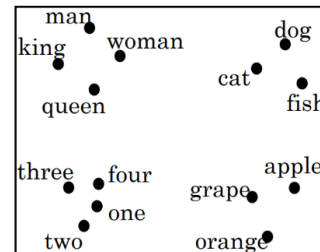
- Find w such that: $\arg \max_w \text{sim}(e_w, e_{\text{king}} - e_{\text{man}} + e_{\text{woman}})$
- 30-75% accuracy at finding the correct answer
- t-SNE takes 300D data and maps in a very non-linear way to a 2D space. Do not expect parallelogram relationships like on the left to hold true



300 D

Find word w : $\arg \max_w$

300D → 2D



t-SNE

$$e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - e_w$$

$$\text{sim}(e_w, e_{\text{king}} - e_{\text{man}} + e_{\text{woman}})$$

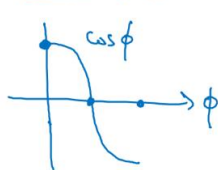
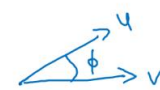
30-75%

- Cosine similarity

- Inner product between u and v ($u^T v$), or you can use squared distance ($\|u-v\|^2$) (measure of dissimilarity). Main difference is normalization for lengths
- Formula is cosine of angle between u and v

$$\rightarrow \text{sim}(e_w, e_{\text{king}} - e_{\text{man}} + e_{\text{woman}})$$

$$\text{sim}(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$$



$$\|u-v\|^2$$

Man:Woman as Boy:Girl

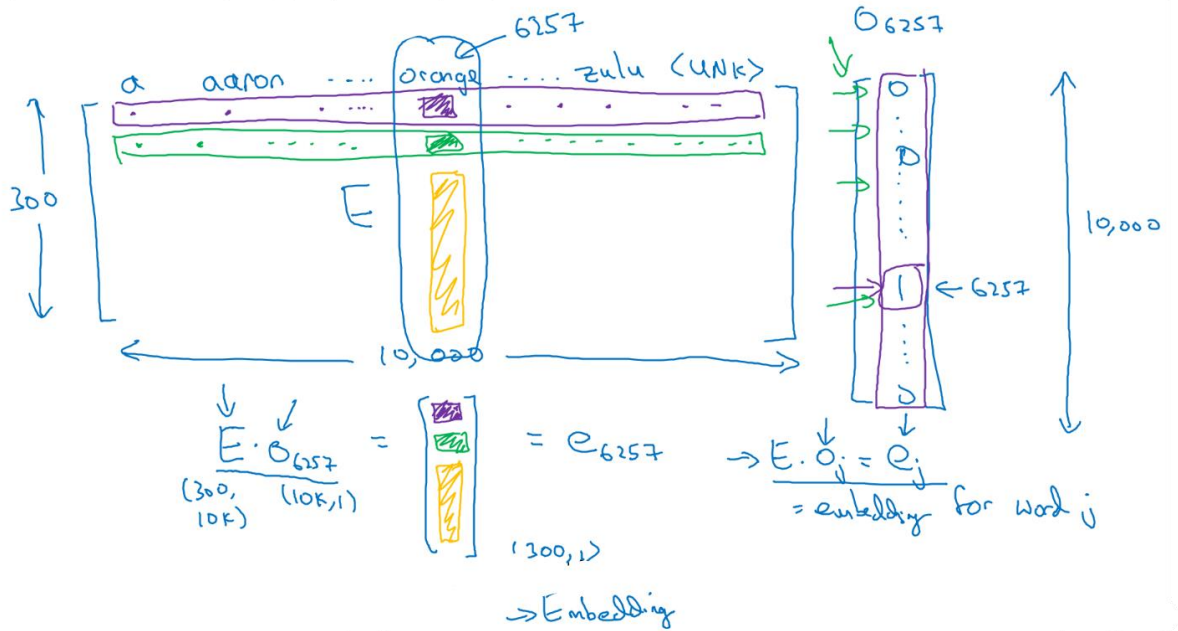
Ottawa:Canada as Nairobi:Kenya

Big:Bigger as Tall:Taller

Yen:Japan as Ruble:Russia

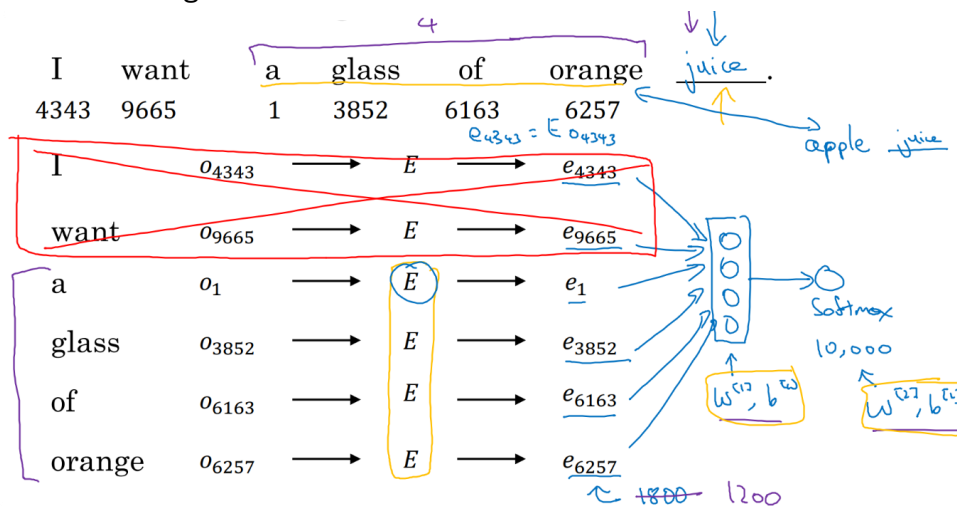
Embedding Matrix

- When you implement an algorithm to learn word embedding, it learns an embedding matrix E
- Embedding for word j : $E * \mathbf{o}_j = \mathbf{e}_j$ (embedding matrix * one-hot vector = embedding)
- In practice, use specialized function to look up an embedding



Learning Word Embeddings

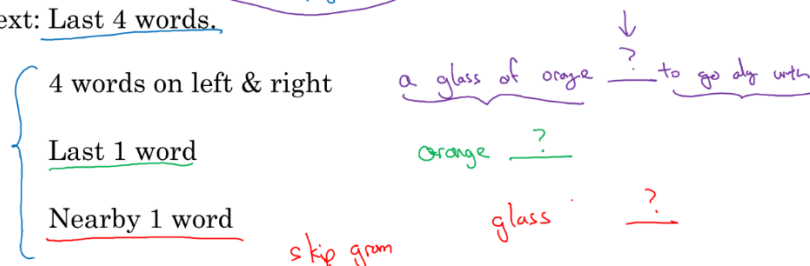
- Neural language model
 - Predicting next word in sequence
 - Building a neural language model is a reasonable way to learn a set of embeddings
 - Input to SoftMax is obtained by stacking embedding words for each word
 - More commonly, fixed historical window of words before the target (e.g. prev 4 words). Then you can deal with even arbitrarily long sentences because the input sizes are always fixed
 - Weights are also parameters of algorithm. Use backprop to perform gradient descent to maximize the likelihood of your training set to just repeatedly predict: given 4 words in a sequence, what is your next word in your text corpus?
 - Algorithm is incentivized to learn similar word embeddings for similar words – fits training set better



- Other context/target pairs
 - Algorithm predicts target word using some context

I want a glass of orange juice to go along with my cereal.

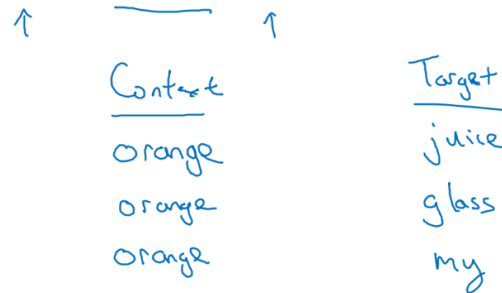
Context: Last 4 words.



Word2Vec

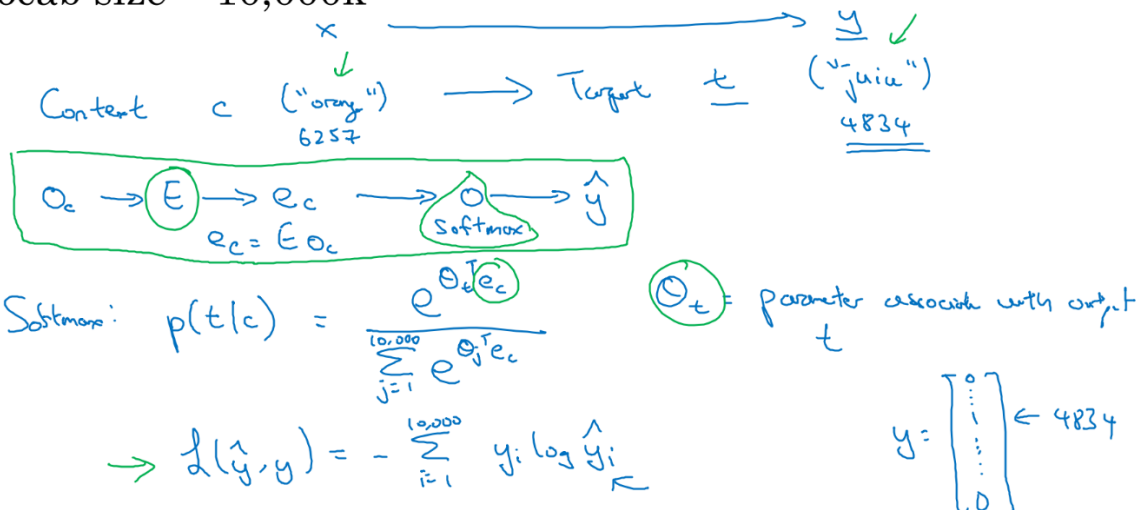
- Simpler and more computationally efficient way to learn embeddings
- **Skip-grams**
 - Create context-to-target pairs: randomly choose the context word. Then randomly choose another word within say +/- 5 words to be the target word.
 - Supervised learning problem: given a context word, predict what is a randomly chosen word within say +/- 10 word window of the input context word
 - Goal of SL problem is not to perform well on the task, but to learn good word embeddings

I want a glass of orange juice to go along with my cereal.



- Skip-gram model
 - Learn mapping from context c to some target t
 - One-hot vector $\rightarrow$ embedding matrix $\rightarrow$ embedding for context word
 - Feed embedding vector e_c to a softmax unit which outputs $\hat{y}$
- Softmax predicts probability of targets given context words. Has parameter θ_t associated with output t
- Loss is $-\sum y_i \log \hat{y}_i$

Vocab size = 10,000k



- Problems with softmax classification
 - Computational speed (or lack thereof) (especially with large vocabulary) of summing over denominator
 - Solution: **hierarchical SoftMax classifier** to split up classifications
 - Is the target word in first 5,000 or second 5,000 of the vocabulary? Etc.
 - Computational cost of hierarchical scales like $\log |V|$
 - In practice, doesn't use a perfectly symmetric tree (i.e. common words on top, less common words deeper)
 - Sampling context c :
 - Uniformly at random from training corpus: can become un-insightfully dominated by frequent words
 - Heuristics to balance out common and uncommon words

$$p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{10,000} e^{\theta_j^T e_c}}$$

Hierarchical softmax. $\log |V|$

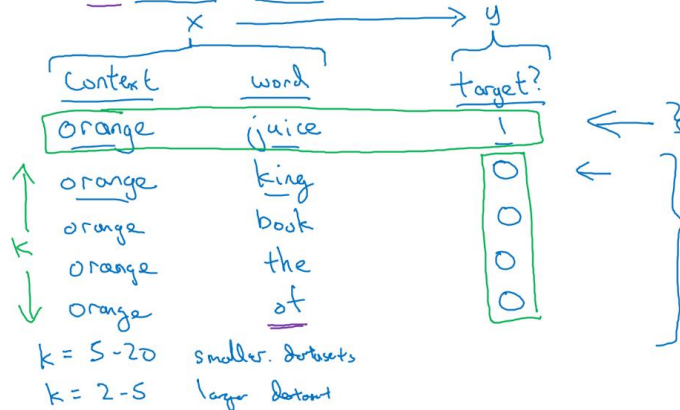
How to sample the context c ?

$\rightarrow$ the, of, a, and, to, ... $c \rightarrow t$
 $\rightarrow$ orange, apple, banana $P(c)$
 $\rightarrow$ duration

Negative Sampling

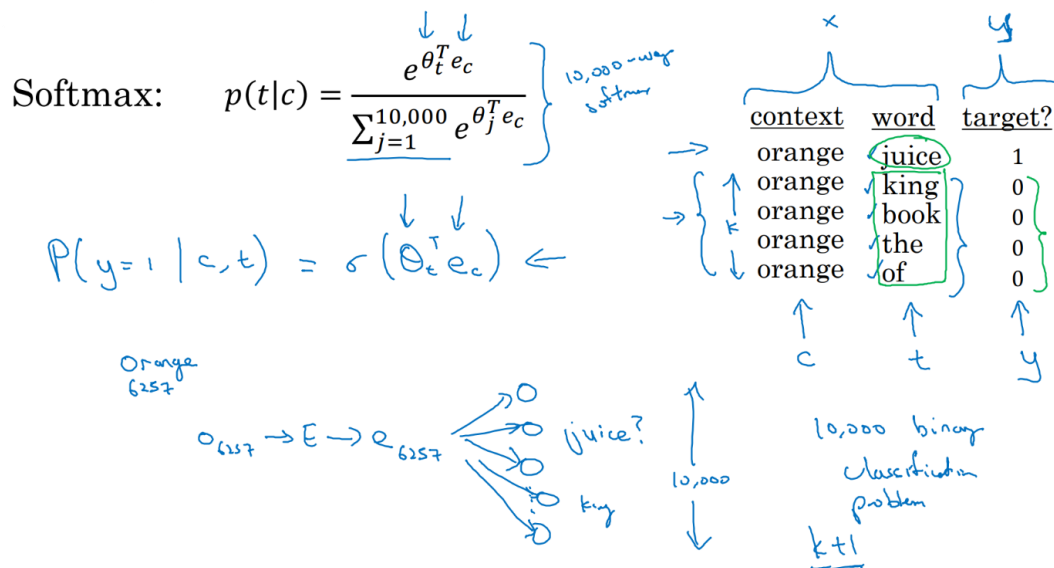
- Similar to skip-gram, but with a much more efficient learning algorithm
- Defining a new learning problem
 - Sample context and target words, label pairs (1 = yes target, 0 = no)
 - Positive examples: target is pulled from window around the text (target = 1)
 - Negative examples: target is random from dictionary (target = 0)
 - It's okay if some of the words randomly chosen from the dictionary are also in the sentence context window
 - Algorithm takes (context, target word) pairs and tries to predict the target label. Do we think the words appear together? Do we think these two words were obtained by sampling two words close to each other?
 - Choose # neg examples $k = 5-20$ for small datasets, $k=2-5$ for larger datasets

I want a glass of orange juice to go along with my cereal.



- Negative sampling model

- Logistic regression model with sigmoid applied to parameter vector θ_t^T for each possible target word and separate embedding vector e_c for each possible context word) to estimate the probability that $y=1$
- k:1 ratio of negative to positive examples
- one-hot vector $\rightarrow$ embedding matrix $\rightarrow$ embedding vector $\rightarrow |V|$ binary classifiers for each possible word
- Instead of training each binary classifier on every iteration, only train k+1: one corresponding to the actual target word and k randomly chosen negative examples. This becomes $|V|$ binary classification problems, each of which is cheap to compute. Each iteration only trains k+1 of them



- Selecting negative examples
 - Sample candidate words according to the empirical frequency in your corpus → overrepresentation of very common words, less representation for harder words
 - Uniformly at random: $1/|V|$ also not representative of distribution
 - In between the two extremes: sample proportional to the frequency of the word

<u>context</u>	<u>word</u>	<u>target?</u>
orange	juice	1
orange	king	0
orange	book	0
orange	the	0
orange	of	0

the, of, and, ...

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=1}^{10,000} f(w_j)^{3/4}} \quad \frac{1}{|V|}$$

Glove Word Vectors (Global Vectors for Word Representation)

- Not as commonly used as Word2Vec and skip-gram, but is simpler
 - x_{ij} = number of times j appears in the context of i (words appear with each other)
 - $x_{ij} = x_{ji}$ for two-sided contexts (before and after), but maybe not for one-sided (before)

I want a glass of orange juice to go along with my cereal.

c, t

x_{ij} = # times j appears in context of i .

$x_{ij} = x_{ji}$ ←

- GloVe model
 - Minimize difference via gradient descent
 - Learn vectors so that their inner product is a good predictor for how often the two words occur together
 - If $x_{ij} = 0$, then $\log x_{ij}$ is undefined, so we don't sum over terms where $x_{ij} = 0$.
 - Add a weighting term $f(x_{ij}) = 0$ if $x_{ij} = 0$ and also gives a meaningful amount of computational to less frequent words and more weight (but not an unduly large amount) to more common words
 - The roles of θ and e are completely symmetric (can swap around). Can take average

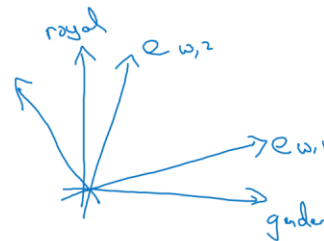
Minimize $\sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(x_{ij}) (\underbrace{\theta_i^T e_j + b_i + b'_j}_{\text{weighting term}} - \log x_{ij})^2$

Annotations:

- $\theta_i^T e_j$ is labeled with t and c arrows, and $\theta_t^T e_c$ is written below it.
- $f(x_{ij}) = 0$ if $x_{ij} = 0$.
- $\log 0 = 0$ (written as $0 \log 0 = 0$).
- θ_i, e_j are symmetric.
- $e_w^{(final)} = \frac{e_w + \theta_w}{2}$
- Example words: this, is, a, ... (with derivation written below).

- Note on the featurization view of word embeddings
 - Cannot guarantee individual components of embeddings are interpretable
 - First feature might be a combination of interpretable features

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)
Gender	-1	1	-0.95	0.97
Royal	0.01	0.02	0.93	0.95
Age	0.03	0.02	0.70	0.69
Food	0.09	0.01	0.02	0.01

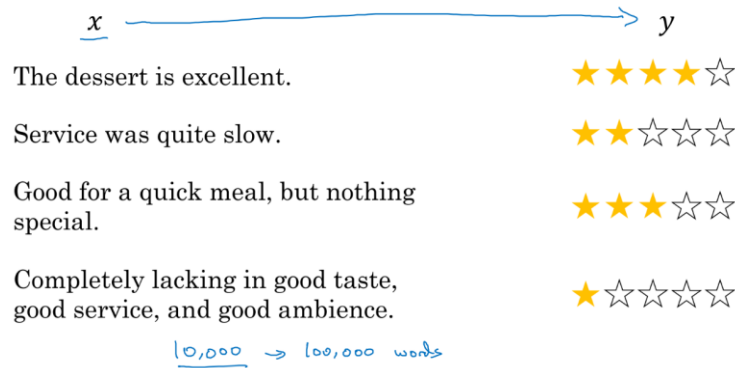


minimize $\sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(x_{ij}) (\theta_i^T e_j + b_i + b'_j - \log x_{ij})^2$

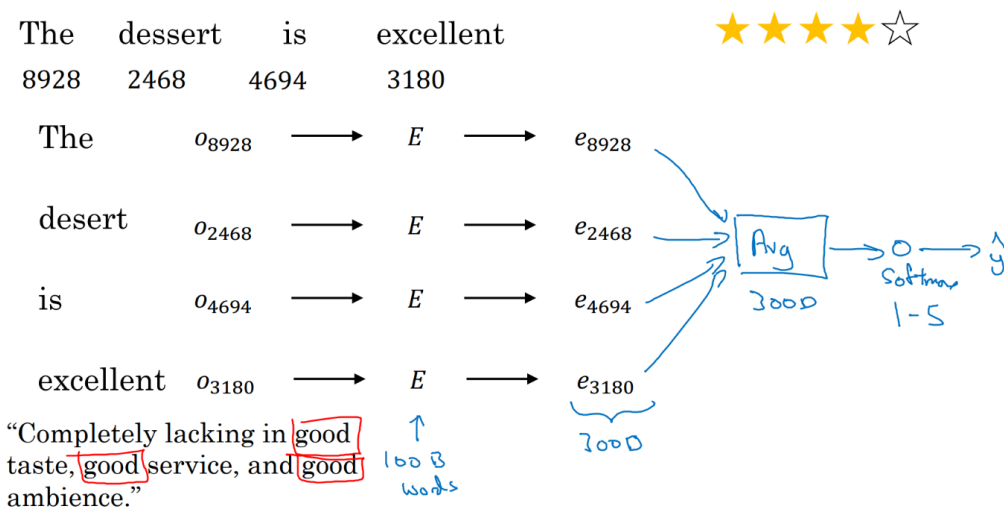
Annotation: $(A\theta_i)^T (A^T e_j) = \theta_i^T A^T A e_j$

Sentiment Classification

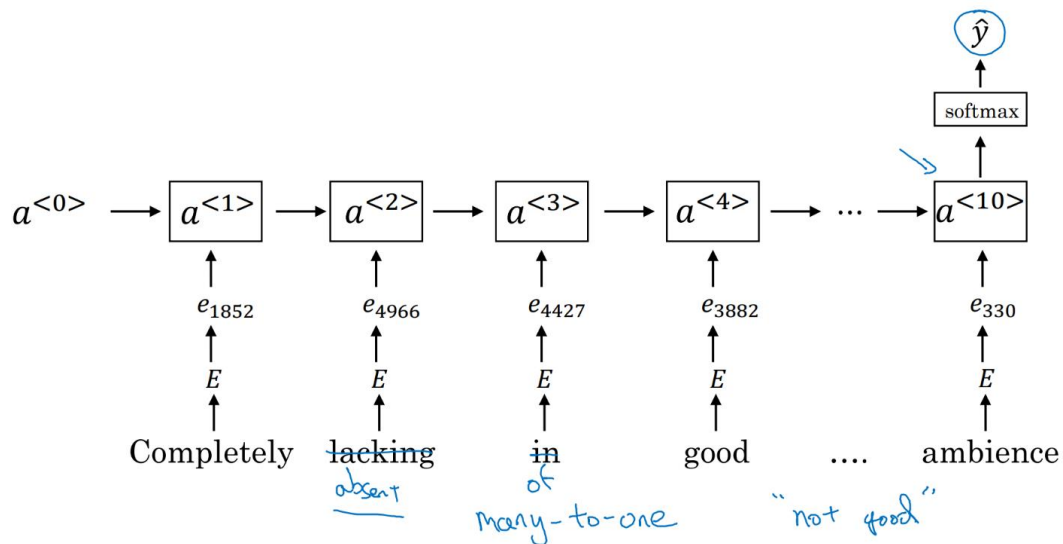
- May not have a huge training set. Use word embedding to build good sentiment classifiers even with modest-size labeled training sets
- Sentiment classification problem



- Simple sentiment classification model
 - Map each word to embedding, sum or average them, and pass into a softmax classifier to put $\hat{y}$ (e.g. probabilities of each star rating)
 - Works for reviews that are short or long
 - However, ignores word order (e.g. “lacking” is not carried over to “good”. Some type of Boolean needs to be applied across)



- RNN for sentiment classification
 - Feed embedding vectors into RNN (many-to-one) and compute a presentation at the last time step to predict $\hat{y}$



Debiasing Word Embeddings

- The problem of bias in word embeddings
 - Input text can be socially biased and the embeddings can pick up on this
 - Since learning algorithms are making important decisions, we need to diminish as much as possible/eliminate undesirable biases

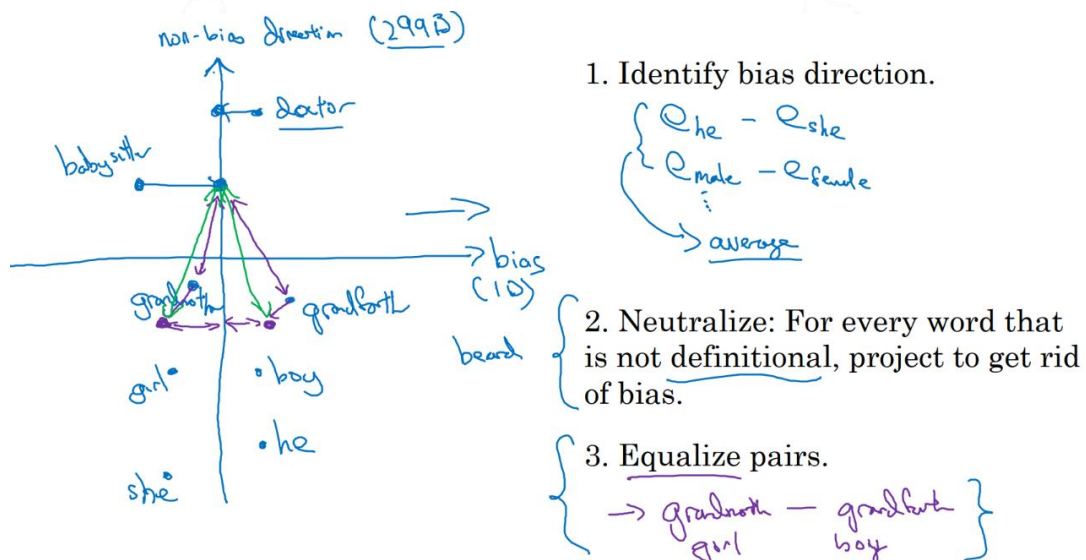
Man:Woman as King:Queen

Man:Computer_Programmer as Woman:Homemaker ✗

Father:Doctor as Mother:Nurse ✗

Word embeddings can reflect gender, ethnicity, age, sexual orientation, and other biases of the text used to train the model.

- Addressing bias in word embeddings
 - 1. Identify bias direction (can be higher than one-dimensional)
 - 2. Neutralize: for every word that is not definitional, project onto axis to get rid of bias
 - 3. Equalize pairs
 - Definitional pairs embeddings should only be different by that factor (e.g. gender), and should not keep any of the bias that was accounted for.
 - Move to a point where they are equally distant from points on axis
 - Authors trained a classifier to determine which words are definitional. Most words in the English language are not definitional in gender
 - Number of pairs to equalize is also small, can usually be handpicked



True/False: Suppose you learn a word embedding for a vocabulary of 60000 words. Then the embedding vectors could be 60000 dimensional, so as to capture the full range of variation and meaning in those words.

True **False**

The dimension of word vectors is usually smaller than the size of the vocabulary.

True/False: t-SNE is a non-linear dimensionality reduction technique.

True False

Suppose you download a pre-trained word embedding which has been trained on a huge corpus of text. You then use this word embedding to train an RNN for a language task of recognizing if someone is happy from a short snippet of text, using a small training set.

x (input text)	y (happy?)
I'm feeling wonderful today!	1
I'm bummed that my cat is ill.	0
Really enjoying this!	1

True/False: Then even if the word “upset” does not appear in your small training set, your RNN might reasonably be expected to recognize “I’m upset” as deserving a label $y = 0$.

True False

Word vectors empower your model with an incredible ability to generalize. The vector for “upset” would contain a negative/unhappy connotation which will probably make your model classify the sentence as a “0”.

Which of these equations do you think should hold for a good word embedding? (Choose all that apply)

$$e_{boy} - e_{brother} \approx e_{sister} - e_{girl}$$

$$e_{boy} - e_{girl} \approx e_{brother} - e_{sister}$$

$$e_{boy} - e_{brother} \approx e_{girl} - e_{sister} \quad \text{Boy is to brother as girl is to sister}$$

$$e_{boy} - e_{girl} \approx e_{sister} - e_{brother}$$

Let E be an embedding matrix, and let o_{1234} be a one-hot vector corresponding to word 1234. Then to get the embedding of word 1234, why don't we call $E * o_{1234}$ in Python?

This doesn't handle unknown words (<UNK>).

None of the above: calling the Python snippet as described above is fine.

The correct formula is $E^T * o_{1234}$.

It is computationally wasteful. element-wise multiplication

When learning word embeddings, we create an artificial task of estimating $P(\text{target} \mid \text{context})$. It is okay if we do poorly on this artificial prediction task; the more important by-product of this task is that we learn a useful set of word embeddings.

True False

True/False: In the word2vec algorithm, you estimate $P(t \mid c)$, where t is the target word and c is a context word. t and c are chosen from the training set to be nearby words.

True False

Suppose you have a 10000 word vocabulary, and are learning 100-dimensional word embeddings. The word2vec model uses the following softmax function:

$$P(t | c) = \frac{e^{\theta_t^T e_c}}{\sum_{t'=1}^{10000} e^{\theta_{t'}^T e_c}}$$

True/False: After training, we should expect θ_t to be very close to e_c when t and c are the same word.

True

False

Suppose you have a 10000 word vocabulary, and are learning 500-dimensional word embeddings. The GloVe model minimizes this objective:

$$\min \sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(X_{ij})(\theta_i^T e_j + b_i + b_j' - \log X_{ij})^2$$

True/False: X_{ij} is the number of times word j appears in the context of word i.

True

False

You have trained word embeddings using a text dataset of t_1 words. You are considering using these word embeddings for a language task, for which you have a separate labeled dataset of t_2 words. Keeping in mind that using word embeddings is a form of transfer learning, under which of these circumstances would you expect the word embeddings to be helpful?

When t_1 is smaller than t_2

When t_1 is larger than t_2

When t_1 is equal to t_2

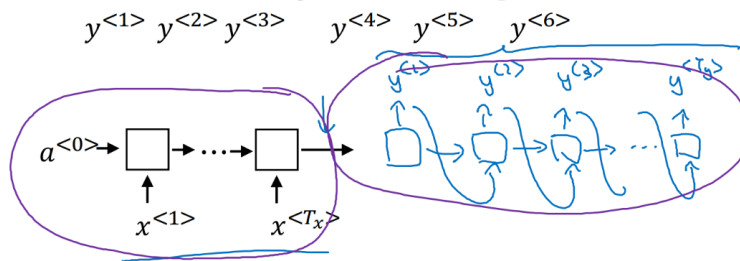
Transfer embeddings to new tasks with smaller training sets.

Basic Sequence to Sequence Models

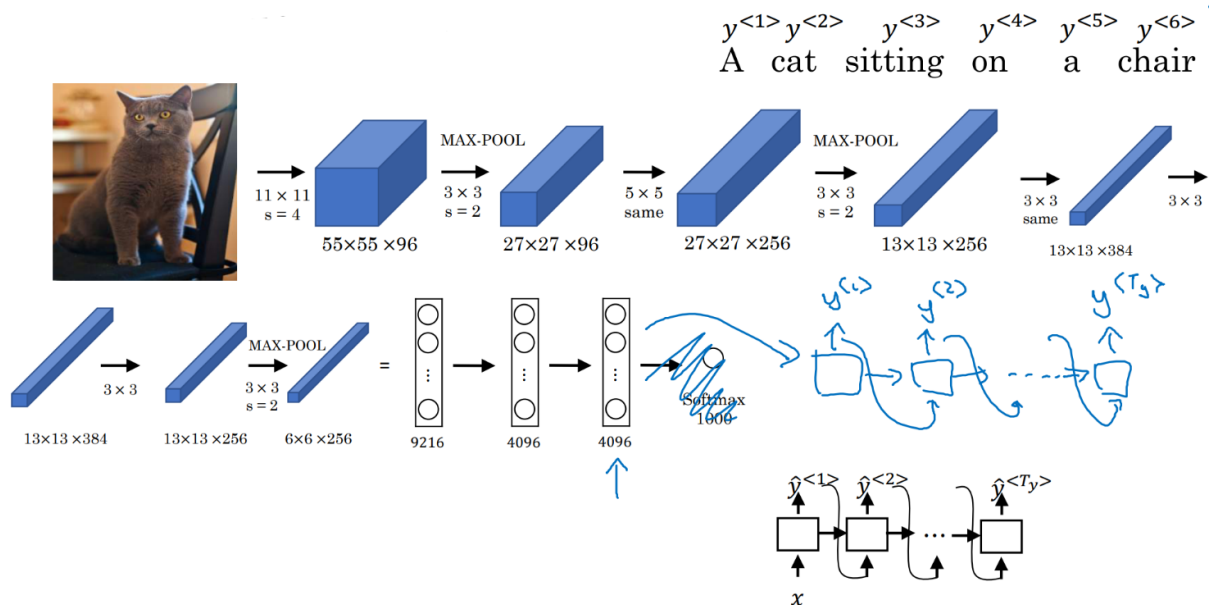
- Sequence to sequence model
 - Build encoder network as an RNN (GRU or LSTM) that takes French words one at a time and outputs a vector that represents the input sentence
 - Build a decoder network that takes the encoding output from the encoder RNN and train to output the translation one word at a time
 - This model works given enough pairs of French and English sentences

$x^{<1>} \quad x^{<2>} \quad x^{<3>} \quad x^{<4>} \quad x^{<5>}$
 Jane visite l'Afrique en septembre

→ Jane is visiting Africa in September.

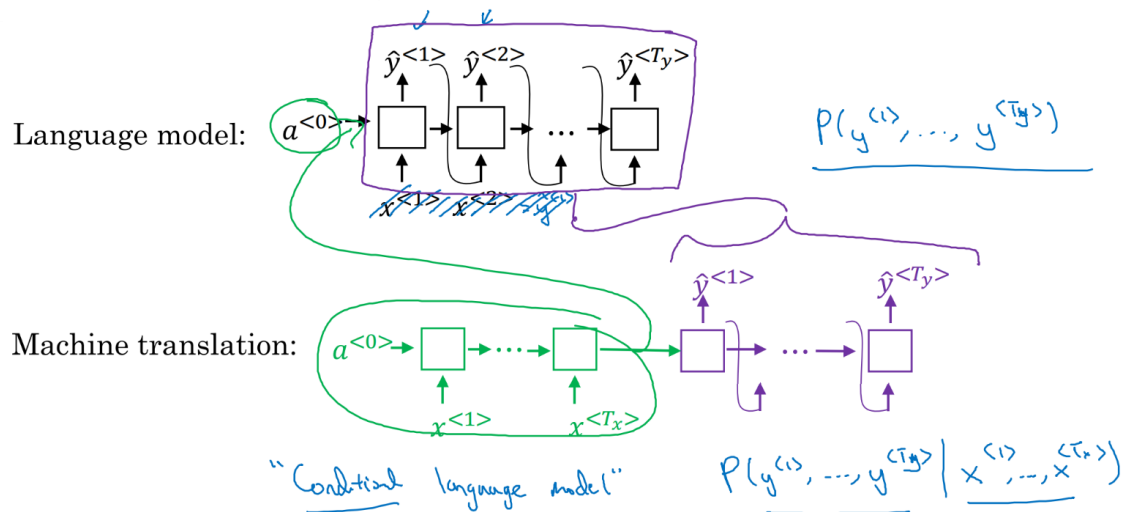


- Image captioning
 - Input image to pre-trained convNet (AlexNet) as an encoder network that gives a 4096-dimensional feature vector representing the image
 - Feed output of convNet to an RNN that generates the caption one word at a time



Picking the Most Likely Sequence

- Machine translation as building a conditional language model
 - Encoder (green) figures out some representation for the input sentence
 - Decoder (purple) resembles the language model
 - Instead of modeling the probability of any sentence, it is now modeling the probability of the output translation conditioned on some input sentence



- Finding the most likely translation
 - Probability of different English translations
 - Do NOT sample at random. You will get some bad translations
 - Instead, find the sentence y that maximizes the conditional probability

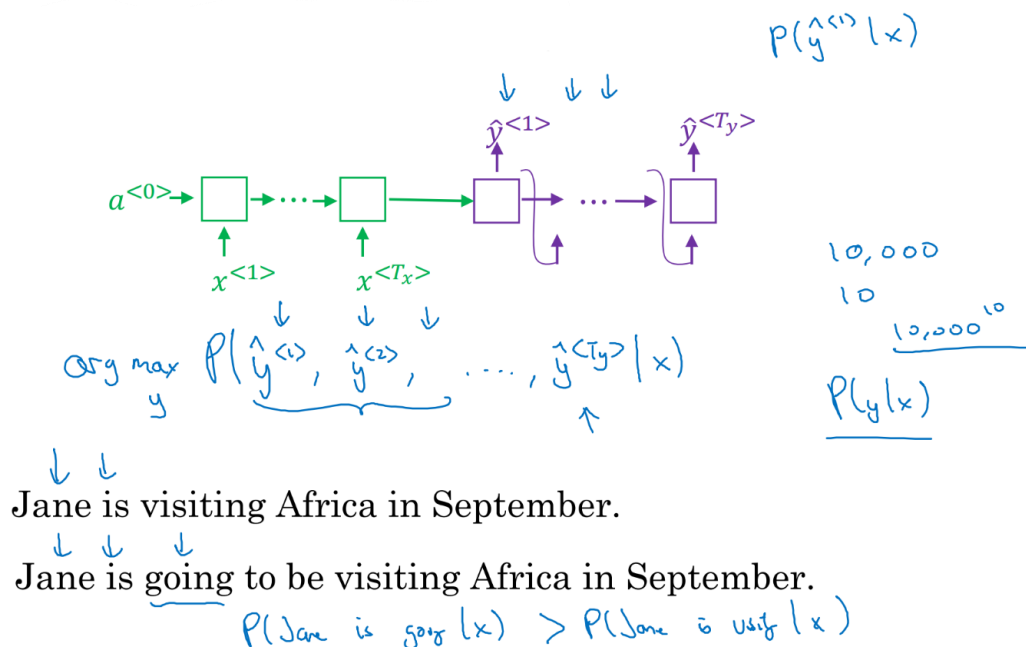
Jane visite l'Afrique en septembre.

$$P(\overset{\text{English}}{y^{<1>}}, \dots, y^{<T_y>} | \overset{\text{French}}{x})$$

- Jane is visiting Africa in September.
- Jane is going to be visiting Africa in September.
- In September, Jane will visit Africa.
- Her African friend welcomed Jane in September.

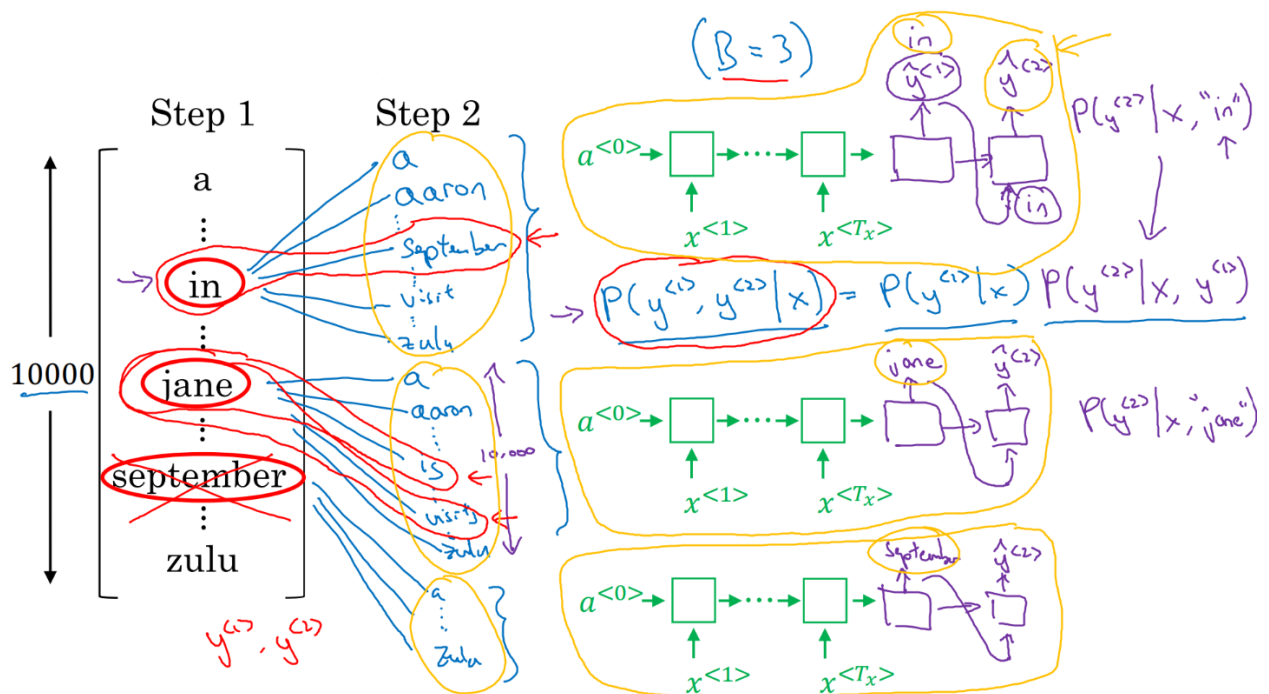
$$\arg \max_{y^{<1>}, \dots, y^{<T_y>}} P(\underline{y^{<1>}}, \dots, y^{<T_y>} | x)$$

- Why not a greedy search?
 - Greedy search: pick whatever word is most likely for each position according to conditional language model (maximize joint probability of the whole sentence)
 - Example: second translation is not a bad translation, but it has more unnecessary words. However, “going” is a more common word than “visiting”, so greedy search would pick the second translation at the 3rd word position
 - Total number of combinations is exponentially large, can’t enumerate
 - Use an approximate search algorithm that tries to pick the sentence y that maximizes the conditional probability. Not guaranteed to find that value of y that maximizes, but it usually does a good job

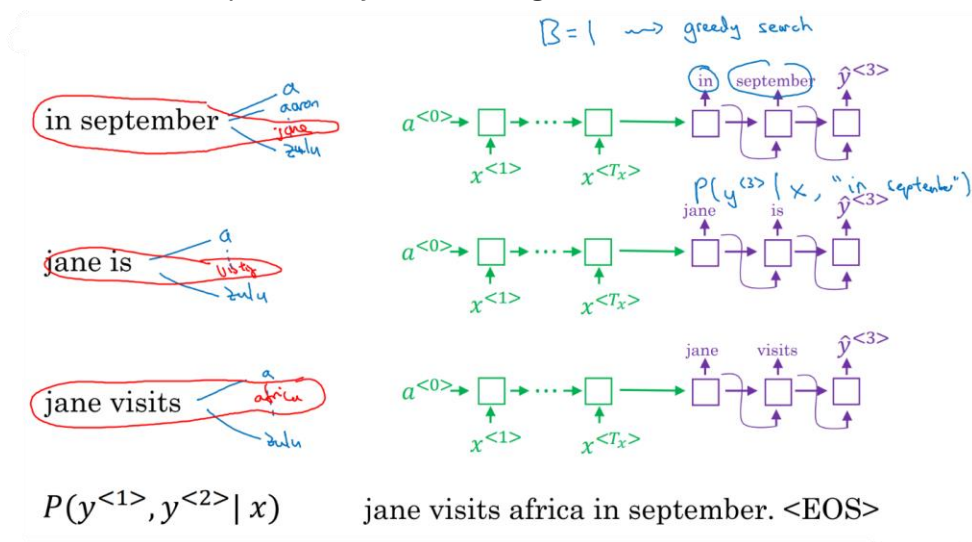


Beam Search

- Given an input audio clip, output most likely text transcript
- Beam search algorithm
 - Use encoder (green) and decoder (purple) to evaluate the probability of the 1st word $P(y^{<1>} | x)$ (SoftMax output over all 10,000 possibilities). Will consider multiple alternatives and store them in memory
 - $B = \text{beam width}$ = number of top candidate words the algorithm considers/stores
 - To evaluate the probability of 2nd word, instantiate B copies of the network and for each, hard wire each of the B candidate 1st words and calculate conditional probability of each candidate 2nd word given the 1st
 - $P(y^{<1>}, y^{<2>} | x) = P(y^{<1>} | x) * P(y^{<2>} | x, y^{<1>})$
 - Number of possibilities considered per layer: $B * |v|$
 - Pick top three for each step. Some candidate previous words may be eliminated in the next step



- Evaluate probability of 3rd word given 1st two words



Refinements to Beam Search

- Length normalization
 - Log of product becomes sum of logs
 - Product of probabilities vanishes when numbers are small, and favors short translations
 - Maximizing sum of logs is better than product of probabilities. Will give the same result (scales but doesn't shift) because log fcn is monotonically increasing
 - Normalize by number of words in translation T_y to get average of log probability for each word. Significantly reduces the penalty for longer translations. Can introduce hyperparameter α to control strength of normalization
 - Normalized log likelihood objective function

$$P(y^{(1)} \dots y^{(T_y)} | x) = \frac{P(y^{(1)} | x, y^{(1)}) \dots P(y^{(T_y)} | x, y^{(1)}, \dots, y^{(T_y-1)})}{P(y^{(1)} | x, y^{(1)}, \dots, y^{(T_y-1)})}$$

$$\arg \max_y \prod_{t=1}^{T_y} P(y^{(t)} | x, y^{(1)}, \dots, y^{(t-1)})$$

$$\arg \max_y \sum_{t=1}^{T_y} \log P(y^{(t)} | x, y^{(1)}, \dots, y^{(t-1)}) \leftarrow$$

$$T_y = 1, 2, 3, \dots, 30.$$

$$\rightarrow \frac{1}{T_y^\alpha} \sum_{t=1}^{T_y} \log P(y^{(t)} | x, y^{(1)}, \dots, y^{(t-1)})$$

$$\alpha = 0.7 \quad \frac{\alpha=1}{\alpha=0}$$

- Beam search discussion
 - Choosing beam width B?
 - Large B: better result, but slower
 - Small B: worse result, but faster
 - For production 10 is typical, 100 is large depending on the application
 - For research, people use up to 1,000 or 3,000
 - Diminishing returns for large B. Huge gain from 1 $\rightarrow$ 10, less for 1k to 3k
 - Unlike exact search algorithms like BFS (Breadth First Search) or DFS (Depth First Search), Beam Search runs faster but is **not guaranteed to find exact maximum** for $\arg \max_v P(y|x)$

Error Analysis on Beam Search

- Error analysis on beam search example
 - Just like getting more training data, increasing beam width B never hurts, but may not by itself improve performance to the desired level
 - $y^* = \text{human}$, $\hat{y} = \text{algorithm}$
 - $P(y^* | x) > P(\hat{y} | x)$: the algorithm does not achieve the outcome with greatest likelihood, the beam search failed to give you the value of y that maximizes the probability $\rightarrow$ revise beam search
 - $P(y^* | x) \leq P(\hat{y} | x)$: the algorithm predicts that the better translation had lower probability $\rightarrow$ revise the RNN
 - If using length normalization, evaluate the optimization objective that takes length norm into account (instead of raw probabilities)

$\rightarrow$ RNN

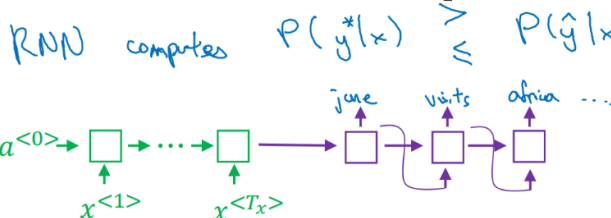
$\rightarrow$ Beam Search

BT

Jane visite l'Afrique en septembre.

Human: Jane visits Africa in September. (y^*)

Algorithm: Jane visited Africa last September. ($\hat{y}$) $\leftarrow$



Human: Jane visits Africa in September. (y^*)

Algorithm: Jane visited Africa last September. ($\hat{y}$)

Case 1: $P(y^* | x) > P(\hat{y} | x) \leftarrow$ $\arg \max_y P(y | x)$

Beam search chose $\hat{y}$. But y^* attains higher $P(y | x)$.

Conclusion: Beam search is at fault.

Case 2: $P(y^* | x) \leq P(\hat{y} | x) \leftarrow$

y^* is a better translation than $\hat{y}$. But RNN predicted $P(y^* | x) < P(\hat{y} | x)$.

Conclusion: RNN model is at fault.

- Error analysis process
 - Go through dev set and find mistakes made by the algorithm
 - Figures out what fraction of errors are “due to” beam search vs RNN model
 - Beam search issue → increase B
 - RNN issue → add regularization, get more training data, try a different network architecture, etc.

Human	Algorithm	$P(y^* x)$	$P(\hat{y} x)$	At fault?
Jane visits Africa in September. - - - ...	Jane visited Africa last September. - - - ...	2×10^{-10} — —	1×10^{-10} — —	(B) (R) B R R ...

Bleu (Bilingual Evaluation Understudy) Score

- Measures accuracy when there are multiple equally good answers
- Measures the degree to which the machine translation is similar to/overlaps with the references
- As long as the machine translation is close to any of the human provided references, it will get a high Bleu score
- Understudy: machine evaluates rather than having humans evaluate every output of the machine translation system
- Look at machine generated output and see if the types of words it generates appear in at least one of the human generated references
- Precision: count if the word appears in the references (arbitrary sentence of “the” repeats has perfect precision)
- **Modified precision:** each word gets credit only up to the maximum number of times it appears in the reference sentences
 - $p = \text{max occurrence in references} / \text{occurrence in machine translation}$

- Evaluating machine translation

French: Le chat est sur le tapis.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: the the the the the the the.

Precision: $\frac{7}{7}$

Modified precision: $\frac{2}{7}$

Handwritten notes:
 - Arrow from "2" to "the" in Reference 1: 2 occurrences
 - Arrow from "2" to "the" in Reference 2: Count_clip("the")
 - Arrow from "7" to "the" in MT output: Count("the")

- Bleu score on bigrams

- Bigrams = pairs of words occurring next to each other
- For each bigram: modified precision p = (max # occurrences in ref 1 or 2) Σ count_clip / Σ bigram counts in machine translation

MT output: The cat the cat on the mat. ←

	Count	Count_clip	
the cat	2 ←	1 ←	
cat the	1 ←	0	4
cat on	1 ←	1 ←	6
on the	1 ←	1 ←	
the mat	1 ←	1 ←	

Handwritten notes:
 - Arrow from "2" to "the cat"
 - Arrow from "1" to "cat the"
 - Arrow from "1" to "cat on"
 - Arrow from "1" to "on the"
 - Arrow from "1" to "the mat"
 - Arrow from "1" to "the mat" (under "the")

- Bleu score on n-grams

- If the MT output is exactly the same as a reference, all $p_n = 1.0$

Example: Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: The cat the cat on the mat. ($\hat{y}$)

$$P_1 = \frac{\sum_{\text{unigram} \in \hat{y}} \text{Count}_{\text{clip}}(\text{unigram})}{\sum_{\text{unigram} \in \hat{y}} \text{Count}(\text{unigram})}$$

$$P_n = \frac{\sum_{\text{n-gram} \in \hat{y}} \text{Count}_{\text{clip}}(\text{n-gram})}{\sum_{\text{n-gram} \in \hat{y}} \text{Count}(\text{n-gram})}$$

Handwritten notes:
 - Arrow from "unigram" to the first equation
 - Arrow from "n-gram" to the second equation

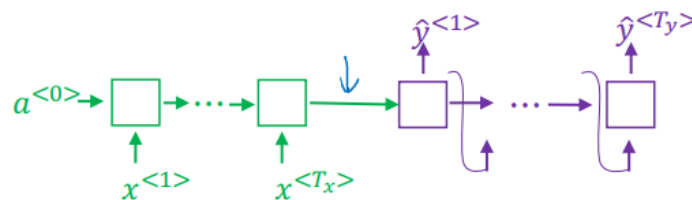
- Bleu details
 - p_n = Bleu score on n-grams only
 - Combined Blue score: $BP \exp(1/n \sum p_n)$
 - BP (brevity penalty) = $\begin{cases} 1 & \text{if MT_output_length} > \text{reference_output_length} \\ \exp(1 - \text{MT_output_length}/\text{reference_output_length}) & \text{otherwise} \end{cases}$

MT system output longer than human generated

penalizes shorter
- Pretty good single real number evaluation metric
- Not used for speech recognition because there is one ground truth, not multiple. Used for image captioning and translation

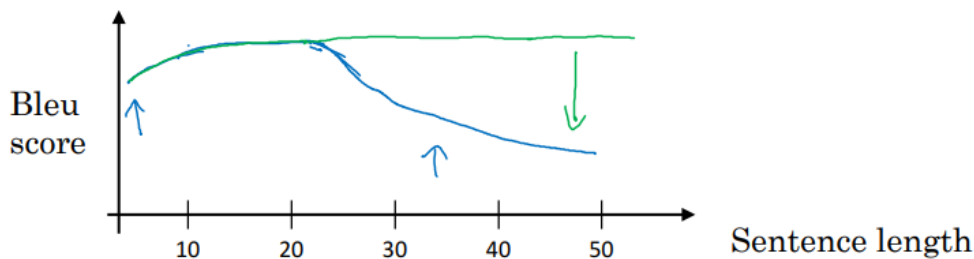
Attention Model Intuition

- The problem of long sequences
 - Asking the encoder network to read in and memorize the whole sentence and store in activations, then feeding to an RNN that outputs a translation all at once. A human translation would read parts and generate translations part-by-part
 - Bleu score works well for short sentences, but performs poorly for long sentences
 - Attention model translates a bit more like humans might (in chunks)

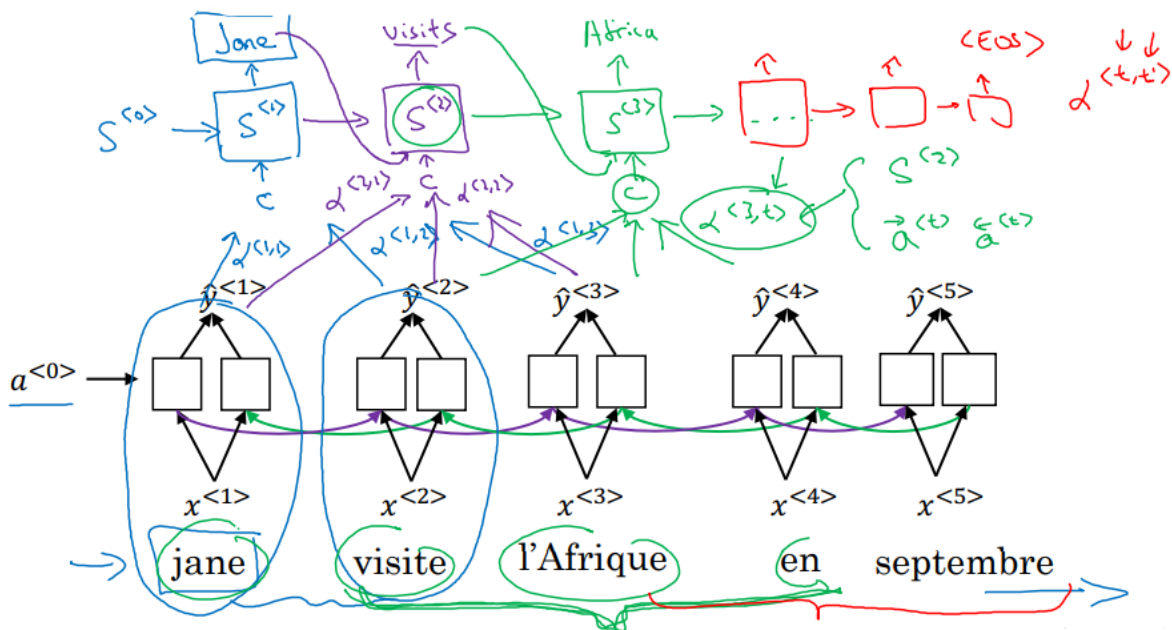


Jane s'est rendue en Afrique en septembre dernier, a apprécié la culture et a rencontré beaucoup de gens merveilleux; elle est revenue en parlant comment son voyage était merveilleux, et elle me tente d'y aller aussi.

Jane went to Africa last September, and enjoyed the culture and met many wonderful people; she came back raving about how wonderful her trip was, and is tempting me to go too.



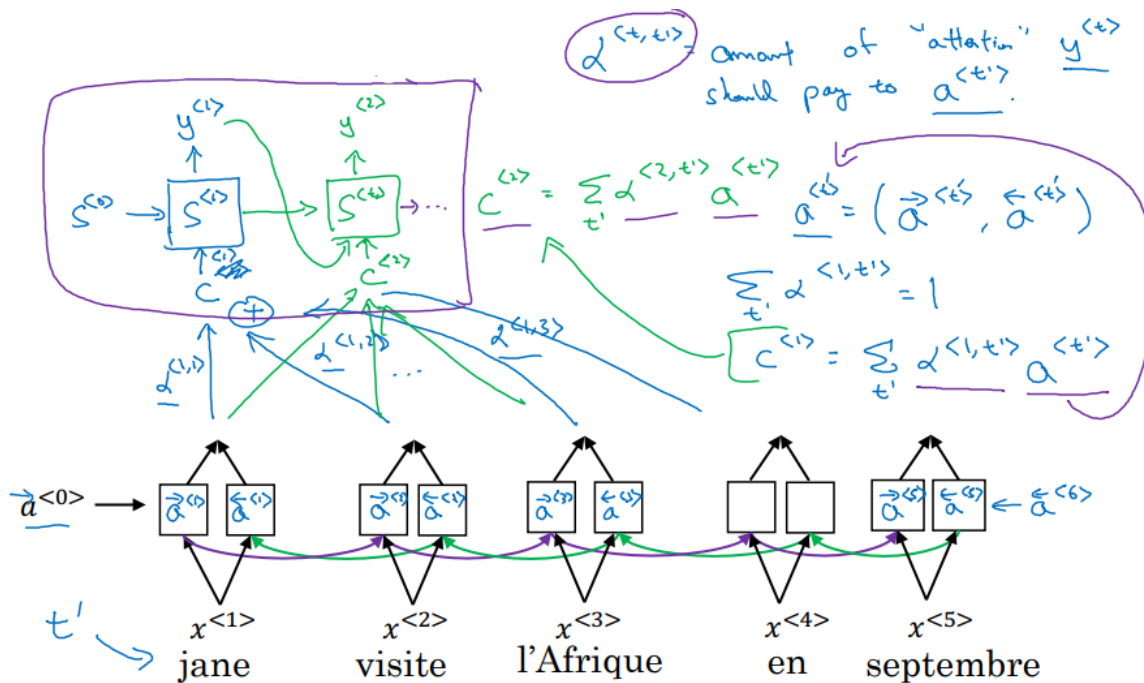
- Attention model intuition
 - Demonstrating with a short sentence even though it was made for long ones
 - Standard bidirectional RNN, but with the $\hat{y}$'s removed since we're not doing a word-for-word translation
 - For each position in the sequence, you can compute a rich set of features about the words in the sentence and may surrounding words
 - Another RNN to generate translations (with activations s for hidden state)
 - Attention model computes a set of **attention weights**
 - $\alpha^{<t, t'>}$ = when generating t^{th} word, how much attention should be paid to word t'
 - Bringing these together gives the context c to pay attention to (c is input to generator RNN)
 - For each step, new set up attention weights
 - The amount each RNN step should be paying attention to the input word at time t is dependent on the activations of the BRNN at time t and state from previous steps



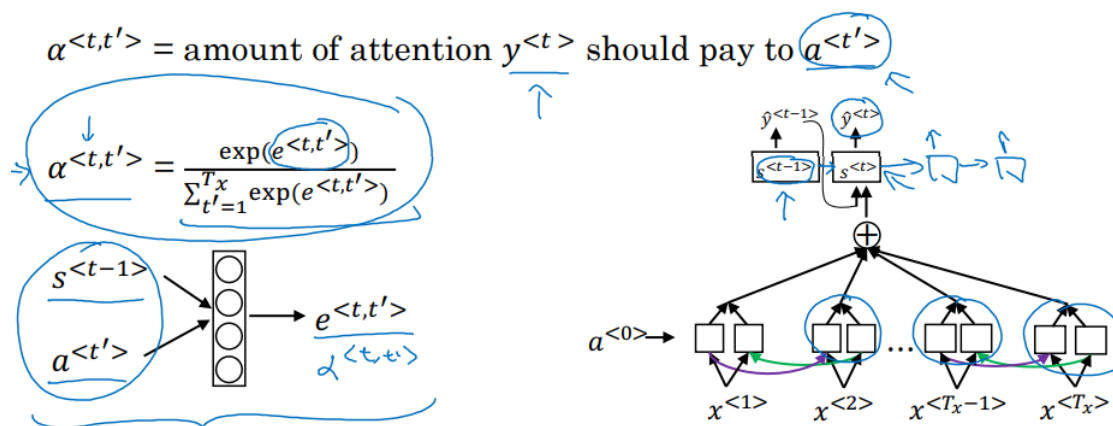
Attention Model

- Model

- $a^{<t'>}$ is the concatenated feature vector for time step t'
- Context c depends on the attention parameters: sum of features from the different time steps $a^{<t'>}$ weighted by the attention weights $\alpha^{<1, t'>}$
- Attention weights are all non-negative and sum to 1
- $\alpha^{<t, t'>}$ = **amount of attention $y^{<t>}$ should pay to $a^{<t'>}$**
- Context vectors are inputs to RNN



- Computing attention $\alpha^{<t,t'>}$
 - Softmax parameterization of $\alpha^{<t,t'>}$ formula ensures they sum to 1
 - Depends on hidden state activation from previous step $s^{<t-1>}$ and features of each position $a^{<t'>}$
 - We don't know what the function is, so compute using small NN (trust backprop and gradient descent to learn the right function)
 - Downside: quadratic cost to run. For T_x words in input and T_y in output, total number of attention parameters is $T_x * T_y$. Can be acceptable for small enough translations

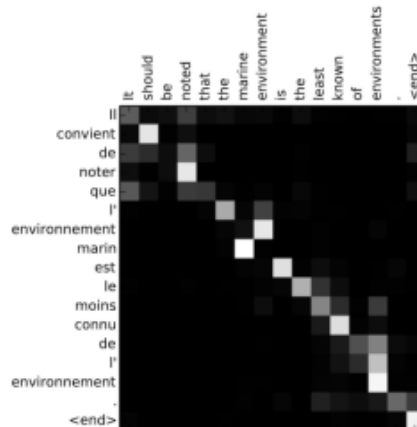


- Attention example: date format normalization

July 20th 1969 $\longrightarrow$ 1969 - 07 - 20

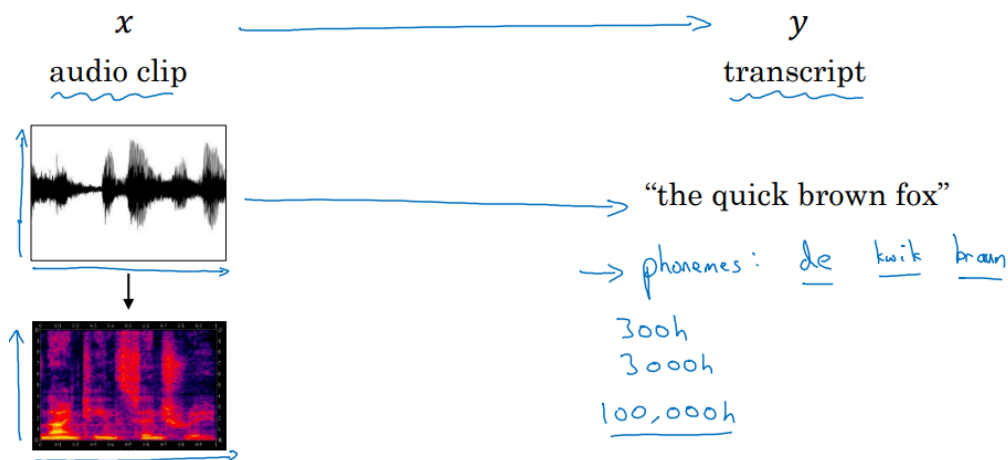
23 April, 1564 $\longrightarrow$ 1564 - 04 - 23

Visualization of $\alpha^{<t,t'>}$:

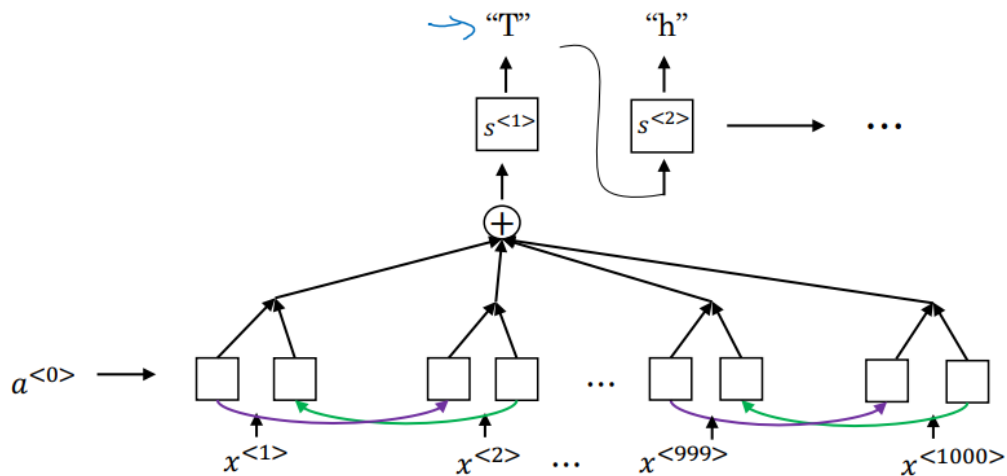


Speech Recognition from Audio Data

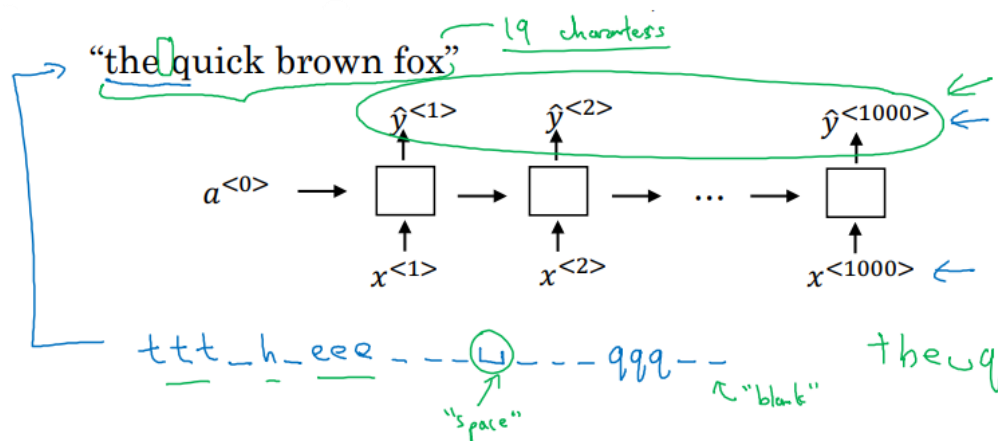
- Speech recognition problem
 - A common pre-processing step is converting each audio file to a spectrograms
 - Human hear does a very similar process to this pre-processing step
 - Phoneme representations are no longer necessary with end-to-end deep learning
 - Very large datasets (academic 300 hr or 3000 hr; commercial 100,000 hr)



- Attention model for speech recognition
 - Horizontal axis is different time frames of the audio input
 - Attention model to output transcript

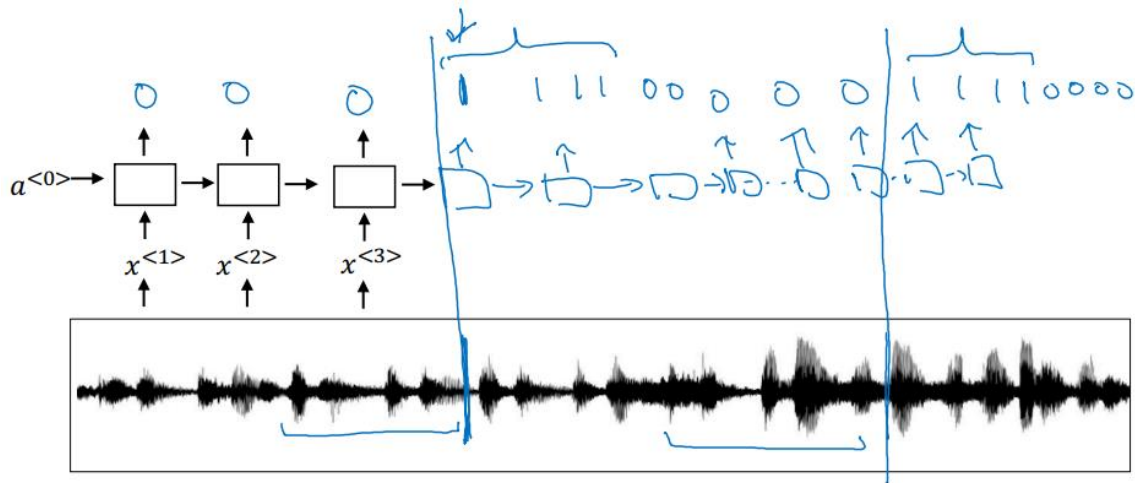


- **CTC (connectionist temporal classification) cost** for speech recognition
 - Here, equal number of input x 's and output y 's
 - In practice, usually a bidirectional LSTM or GRU and deeper
 - In speech recognition, the number of input time steps is usually much bigger than the number of output time steps (e.g. 10 sec clip * 100 Hz = 1000 inputs, but output may not have 1000 characters)
 - Basic rule: collapse repeated characters not separated by "blank"
 - E.g. `ttt_h_eee____qqq_` → the q
 - This representation allows the network to output 1000 characters for smaller intended final outputs (so that the network can be constructed with a large capacity but still function for more smaller amounts)

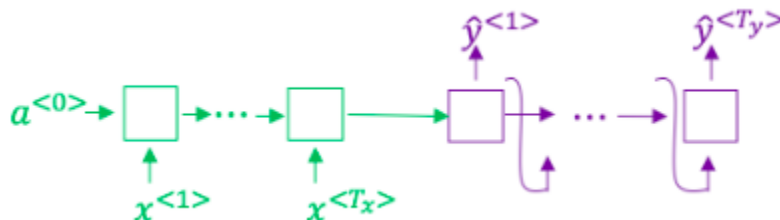


Trigger Word Detection

- Wake up words
- Trigger word detection algorithm
 - Literature is still evolving, this is one example
 - Target label = 0 before trigger word, target label = 1 when said
 - If said again, set to 1 and set to zero before
 - Disadvantage: imbalanced training set (more 0s than 1s)
 - Slightly even out by instead of setting a single time step = 1, set to 1 for a few time steps after before reverting back to 0



Consider using this encoder-decoder model for machine translation.



True/False: This model is a “conditional language model” in the sense that the encoder portion (shown in green) is modeling the probability of the input sentence x .

False True

The encoder-decoder model for machine translation models the probability of the output sentence y conditioned on the input sentence x . The encoder portion is shown in green, while the decoder portion is shown in purple.

In beam search, if you decrease the beam width B , which of the following would you expect to be true? Select all that apply.

Beam search will converge after fewer steps.

Beam search will generally find better solutions (i.e. do a better job maximizing $P(y|x)$).

Beam search will run more quickly.

Beam search will use up more memory.

As the beam width decreases, beam search runs more quickly, uses up less memory, and converges after fewer steps, but will generally not find the maximum $P(y|x)$.

True/False: In machine translation, if we carry out beam search using sentence normalization, the algorithm will tend to output overly short translations.

False True

In machine translation, if we carry out beam search without using sentence normalization, the algorithm will tend to output overly short translations

Suppose you are building a speech recognition system, which uses an RNN model to map from audio clip x to a text transcript y . Your algorithm uses beam search to try to find the value of y that maximizes $P(y | x)$.

On a dev set example, given an input audio clip, your algorithm outputs the transcript $\hat{y}$ = “I’m building an A Eye system in Silly con Valley.”, whereas a human gives a much superior transcript y^* = “I’m building an AI system in Silicon Valley.”

According to your model,

$$P(\hat{y} | x) = 7.21 \cdot 10^{-8}$$

$$P(y^* | x) = 1.09 \cdot 10^{-7}$$

Would you expect increasing the beam width B to help correct this example?

Yes, because $P(y^* | x) > P(\hat{y} | x)$ indicates the error should be attributed to the search algorithm rather than to the RNN.

No, because $P(y^* | x) > P(\hat{y} | x)$ indicates the error should be attributed to the RNN rather than to the search algorithm.

Yes, because $P(y^* | x) > P(\hat{y} | x)$ indicates the error should be attributed to the RNN rather than to the search algorithm.

No, because $P(y^* | x) > P(\hat{y} | x)$ indicates the error should be attributed to the search algorithm rather than the RNN.

$P(y^* | x) > P(\hat{y} | x)$: the algorithm does not achieve the outcome with greatest likelihood, the beam search failed to give you the value of y that maximizes the probability → [revise beam search](#)

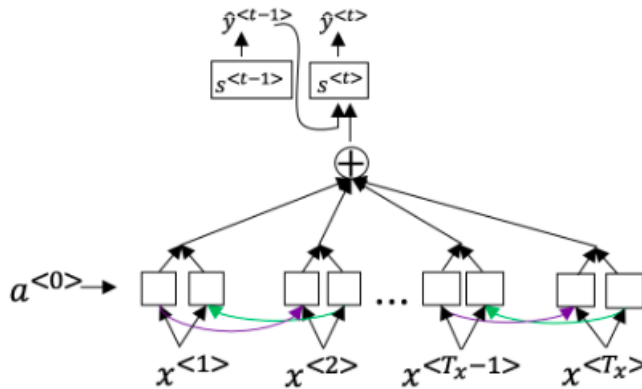
Suppose you are building a speech recognition system, which uses an RNN model to map from audio clip x to a text transcript y . Your algorithm uses beam search to try to find the value of y that maximizes $P(y | x)$.

Suppose you work on your algorithm for a few more weeks, and now find that for the vast majority of examples on which your algorithm makes a mistake, $P(y^* | x) > P(\hat{y} | x)$. This suggests you should not focus your attention on improving the search algorithm.

True **False**

$P(y^* | x) > P(\hat{y} | x)$ indicates the error should be attributed to the search algorithm rather than to the RNN.

Consider the attention model for machine translation.



Further, here is the formula for $\alpha^{<t,t'>}$.

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

Which of the following statements about $\alpha^{<t,t'>}$ are true? Choose all that apply.

$$\sum_{t'} \alpha^{<t,t'>} = 0$$

$$\sum_{t'} \alpha^{<t,t'>} = -1$$

$\alpha^{<t,t'>}$ is equal to the amount of attention $y^{<t>}$ should pay to $a^{<t'>}$

We expect $\alpha^{<t,t'>}$ to be generally larger for values of $a^{<t'>}$ that are highly relevant to the value the network should output for $y^{<t'>}$. (Note the indices in the superscripts.)

$$\sum_{t'} \alpha^{<t,t'>} = 1. \text{ The last answer would be correct if it said for output } y^{<t>}$$

The network learns where to “pay attention” by learning the values $e^{<t,t'>}$, which are computed using a small neural network. Which of the following does $s^{<t>}$ depend on? Select all that apply.

$e^{<t,t'>}$

$\alpha^{<t,t'>}$

s^t is independent of $\alpha^{<t,t'>}$ and $e^{<t,t'>}$.

$s^{<t+1>}$

$s^{<t>}$ depends on $\alpha^{<t,t'>}$ which in turn depends on $e^{<t,t'>}$

The attention model performs the same as the encoder-decoder model, no matter the sentence length.

False True

The performance of the encoder-decoder model declines as the amount of words increases. The attention model has the greatest advantage when the input sequence length T_x is large.

Under the CTC model, identical repeated characters not separated by the “blank” character (_) are collapsed. Under the CTC model, what does the following string collapse to?

kk_eee__ee_p__eeeeeeee__rrrr

keper

keeper

kkeeeeeepeeeeeeeeerrrr

ke epe r

The basic rule for the CTC cost function is to collapse repeated characters not separated by "blank". If a character is repeated, but separated by a “blank”, it is included in the string.

In trigger word detection, $x^{<t>}$ is:

Features of the audio (such as spectrogram features) at time t.

The t-th input word, represented as either a one-hot vector or a word embedding.

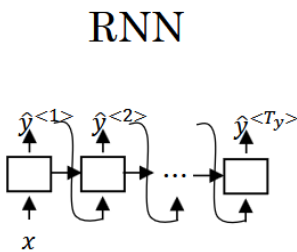
Whether the trigger word is being said at time t.

Whether someone has just finished saying the trigger word at time t.

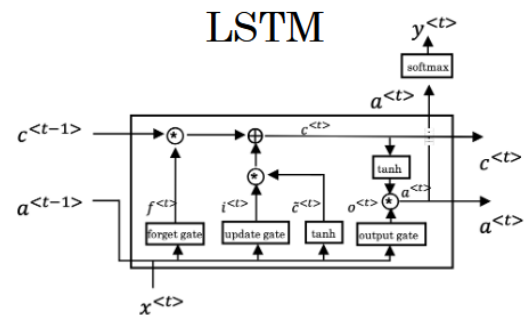
Transformers Intuition

- Motivation
 - RNN has vanishing gradients, which makes it hard to capture long range dependencies in sequences
 - GRU and LSTM address this problem by making use of gates to control flow of information, but this comes with increased complexity
 - All sequential models in that they ingest the input sequence piece-by-piece. Each unit is a bottleneck to the flow of information, since for an output you need the outputs of all units before
 - Transformers allow you to run a lot more of the computations for an entire sequence in parallel

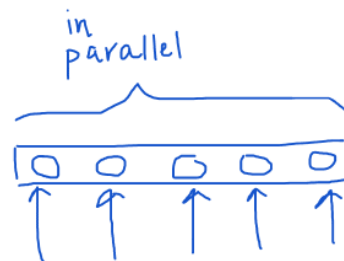
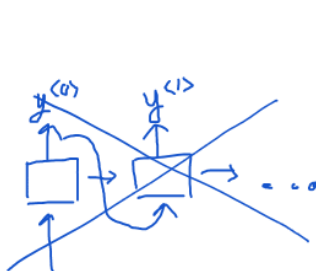
Increased complexity,
sequential



GRU



- Intuition
 - Combines attention representation and CNN-style processing
 - RNN processes one output at a time sequentially
 - CNN takes lots of input at once and computes representations in parallel
 - Attention network computes rich representation with parallel processing
 - **Self-Attention**: attention-based parallel computation of all words in sequence
 - **Multi-Head Attention**: “for loop” over self-attention process to get multiple versions



Self-Attention

- $A(q, K, V)$ = attention-based vector representation of a word
 - Calculate for each word in parallel
 - Figures out context/how the word is meant
 - Also involves a softmax, exponent terms are akin to attention values
 - Q = query, k = key, and v = value

RNN Attention

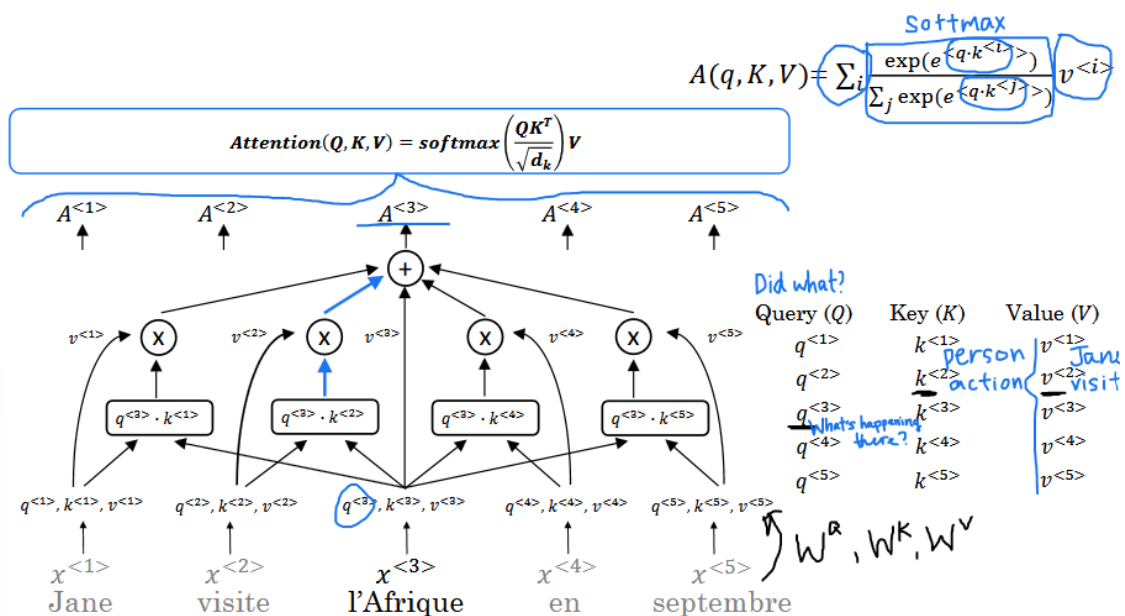
$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^T x \exp(e^{<t,t'>})}$$

Transformers Attention

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

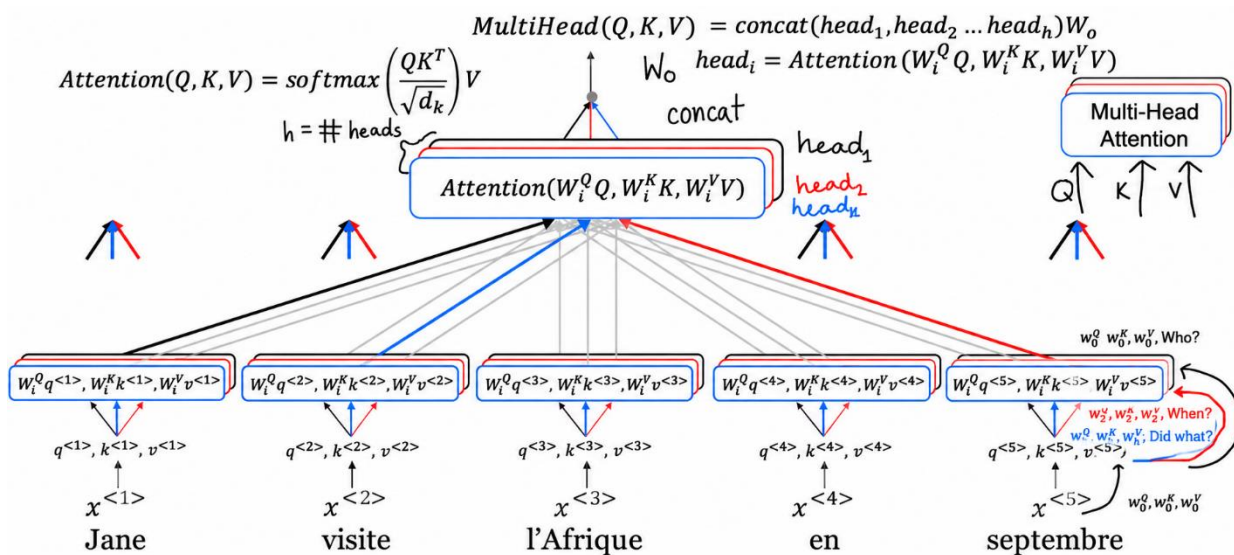
Network

- Parameters: $q^{<j>} = W^Q x^{<j>}$ $k = W^K x^{<j>}$ $v = W^V x^{<j>}$
- Named after database queries and (key, value) pairs
- Inner product between $q^{<j>}$ and $k^{<i>}$ = how good is word i at telling what is happening at word $j \rightarrow$ helps compute the most useful representation $A^{<j>}$
 - E.g. visite gives best context of Africa as travel destination
- Compute softmax over inner products of $q^{<j>}$ (j fixed) and $k^{<i>}$, multiply softmax values times $v^{<i>}$, and sum them up to get $A^{<j>} = A(q^{<j>}, k, v)$
- Advantage: word representations aren't fixed; richer representations of words
- Scaled dot product attention: $A(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$
 - Computations for each word put together
 - Vectorized representation of exp equation above
 - $\sqrt{d_k}$ scales dot product so it doesn't explode

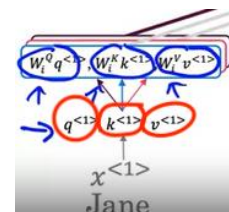


Multi-Head Attention

- “For loop” (can actually be computed in parallel) over self-attention
- Head = each time you calculate self-attention over a sequence
- h = number of heads
- Query key and value vectors as inputs, calculate multiple self-attentions
- Head 1: Weight matrices W_1^Q, W_1^K, W_1^V are learned to determine what’s happening
 - Key word 2 (visite) has the highest inner product with l’Afrique query
- Head 2: W_2^Q, W_2^K, W_2^V answer when something is happening with the query
 - Key word 5 (septembre) has the highest inner product with l’Afrique query
- Head 3: W_3^Q, W_3^K, W_3^V answer who has something to do with the query
 - Key word 1 (Jane) has the highest inner product with l’Afrique query
- Output of the multi-headed attention network = concatenate the h heads and multiply by a weight matrix W_o
- Heads are features that you can pass to a new network to calculate a very rich representation of the sentence

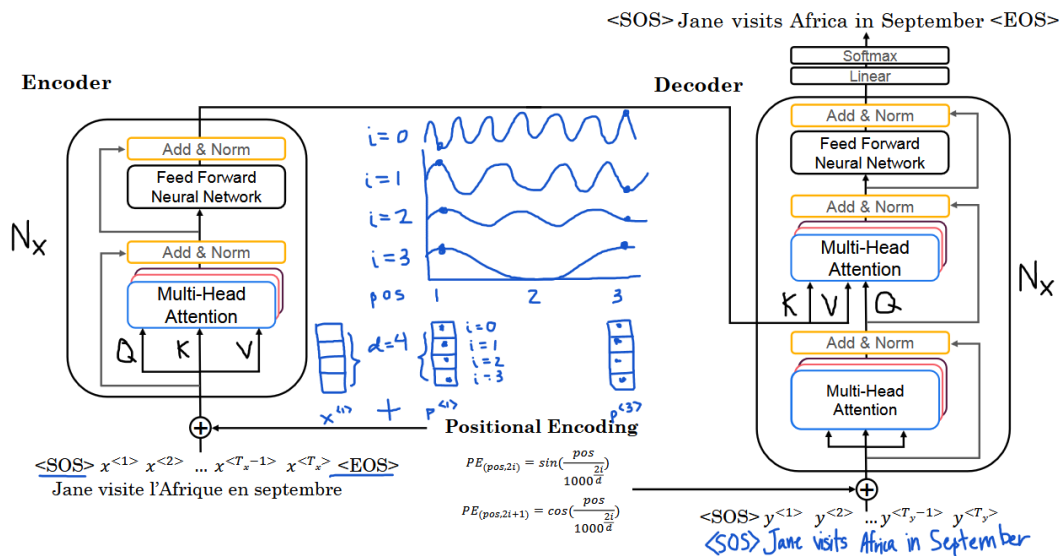


- Note: Previously calculated q, k and v by multiplying x with matrices W . Don’t need to do this for multi-head attention, as you already have the matrices W_i in each head, and you would effectively do the calculation twice if you did the multiplication here also. In the simplest case of multi-headed self-attention you would actually use $q = k = v = x$. The reason we anyway show q, k and v here as different values is that in one part of the transformer (where you calculate the attention between the input and output) the q, k and v are not all the same, as they carry different information.



Transformer Networks

- Added start and end of sentence tokens
- Word embeddings get fed into an encoder block
 - Multi-head attention layer produces a matrix fed into
 - Feed-forward network determines what interesting features are in the sentence
 - Encoding block is repeated N times (typically 6)
- Output of encoder is fed into decoder block
 - Outputs next word in translation: <SOS> ...
 - At each step, the decoder block will input the first few words (whatever we've already generated of the translation)
 - Multi-head attention block uses existing token(s) (<SOS> ...) to compute K, V and outputs the Q matrix for the next multi-head attention block
 - Next multi-head attention block also takes output of encoder to generate K, V (start of English + context from K, V from French to decide what comes next)
 - Feed forward neural network
 - Decoder block also repeated N times (typically 6), output is fed back as input
- Final output layer of linear and softmax to predict next word one at a time
- Simultaneous computation



- Positional encoding of input: creates unique position vector for each word, added as input to encoder
- Residual connections to pass along positional information throughout architecture
- Add & norm layer is similar to batch norm
- Masked multi-headed attention hides that last part to see if, given a perfect first part of translation, the network can predict the next word accurately

A Transformer Network, like its predecessors RNNs, GRUs and LSTMs, can process information one word at a time (Sequential architecture).

True

False

A Transformer Network can ingest entire sentences all at the same time.

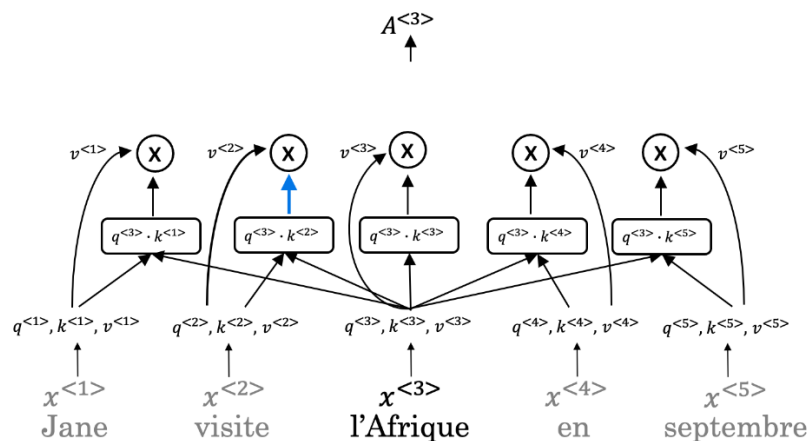
The major innovation of the transformer architecture is combining the use of LSTMs and RNN sequential processing.

True

False

The major innovation of the transformer architecture is combining the use of attention based representations and a CNN convolutional neural network style of processing.

How does the Self-Attention mechanism of transformers use neighboring words to compute a word's context?



Selecting the minimum word values to map the Attention related to that given word.

Summation of the word values to map the Attention related to that given word.

Multiplication of the word values to map the Attention related to that given word.

Selecting the maximum word values to map the Attention related to that given word.

Given a word, its neighboring words are used to compute its context by summing up the word values to map the Attention related to that given word.

Which of the following correctly represents *Attention*?

$$Attention(Q, K, V) = \min \left(\frac{QV^T}{\sqrt{d_k}} \right) K$$

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QV^T}{\sqrt{d_k}} \right) K$$

$$Attention(Q, K, V) = \min \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Are the following statements true regarding Query (Q), Key (K) and Value (V) ?

Q = interesting questions about the words in a sentence

K = specific representations of words given a Q

V = qualities of words given a Q

False True

Q = interesting questions about the words in a sentence,

K = qualities of words given a Q, V = specific representations of words given a Q

$$Attention(W_i^Q Q, W_i^K K, W_i^V V)$$

What does *i* represent in this multi-head attention computation?

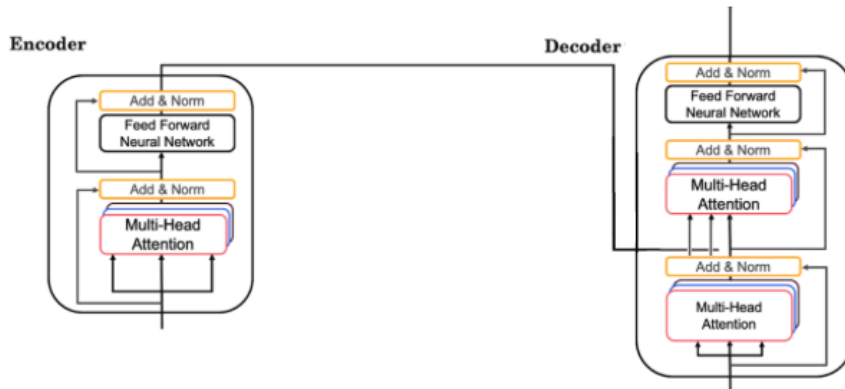
The computed attention weight matrix associated with the *ith* “word” in a sentence.

The computed attention weight matrix associated with the *ith* “head” (sequence)

The computed attention weight matrix associated with specific representations of words given a Q

The computed attention weight matrix associated with the order of the words in a sentence

Following is the architecture within a Transformer Network (*without displaying positional encoding and output layers(s)*).



What is generated from the output of the *Decoder's* first block of *Multi-Head Attention*?

Q **V** **K**

This first block's output is used to generate the Q matrix for the next Multi-Head Attention block.

The output of the decoder block contains a softmax layer followed by a linear layer to predict the next word one word at a time.

False **True**

The output of the decoder block contains a linear layer followed by a softmax layer to predict the next word one word at a time.

Why is positional encoding important in the translation process? (Choose all that apply)

Position and word order are essential in sentence construction of any language.

It helps to locate every word within a sentence.

It is used in CNN and works well there.

Providing extra information to our model.

Which of these is **not** a good criterion for a good positional encoding algorithm?

The algorithm should be able to generalize to longer sentences.

It must be deterministic.

Distance between any two time-steps should be consistent for all sentence lengths.

It should output a common encoding for each time-step (word's position in a sentence).